

実験ログ: 毛とリグ (グルーム・APEX)

2026-09-24 | 収録 13 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- ・毛を生やす — density は本数ではなく「面積あたりの本数」だった
- ・毛を束ねる — つなぐ順番が、思っていたのと逆だった
- ・つないだのに、ガイドが1本も使われていなかった — rest が (0,0,0) だった
- ・土台が動いたとき、毛は付いてくるのか — 入力2をつなぐかどうかだけの差
- ・APEX に入る — 2,220個の部品と、いちばん小さいグラフ
- ・骨を作って動かす — rotate と prerotate を取り違えると、関節ごと飛んでいく
- ・骨に肉を付ける — 曲げると痩せる。90度で 27% 減った
- ・痩せない曲げ方はあった — Dual Quaternion で体積の減りが 154分の1 になる
- ・なぜ Linear が既定なのか — 速さではなかった。差は 4%
- ・逆運動学 (IK) — 届かない場所を指すと、伸びきってそこで止まる
- ・長い鎖の IK — 式は同じ。ただし解き方は2種類あり、得意な範囲が逆
- ・APEX のグラフを持ち歩く — ネットワークが、そのままジオメトリになる
- ・hairgen の毛の本数は $\text{Density} \times \text{面積}$ — 1本は Segments + 1 点、長さは Length ちょうど

毛を生やす — density

は本数ではなく「面積あたりの本数」だった

2026-09-24 | Houdini 21.0.700 | /obj/groom | ノード 3 / 接続 2

何を作ったか

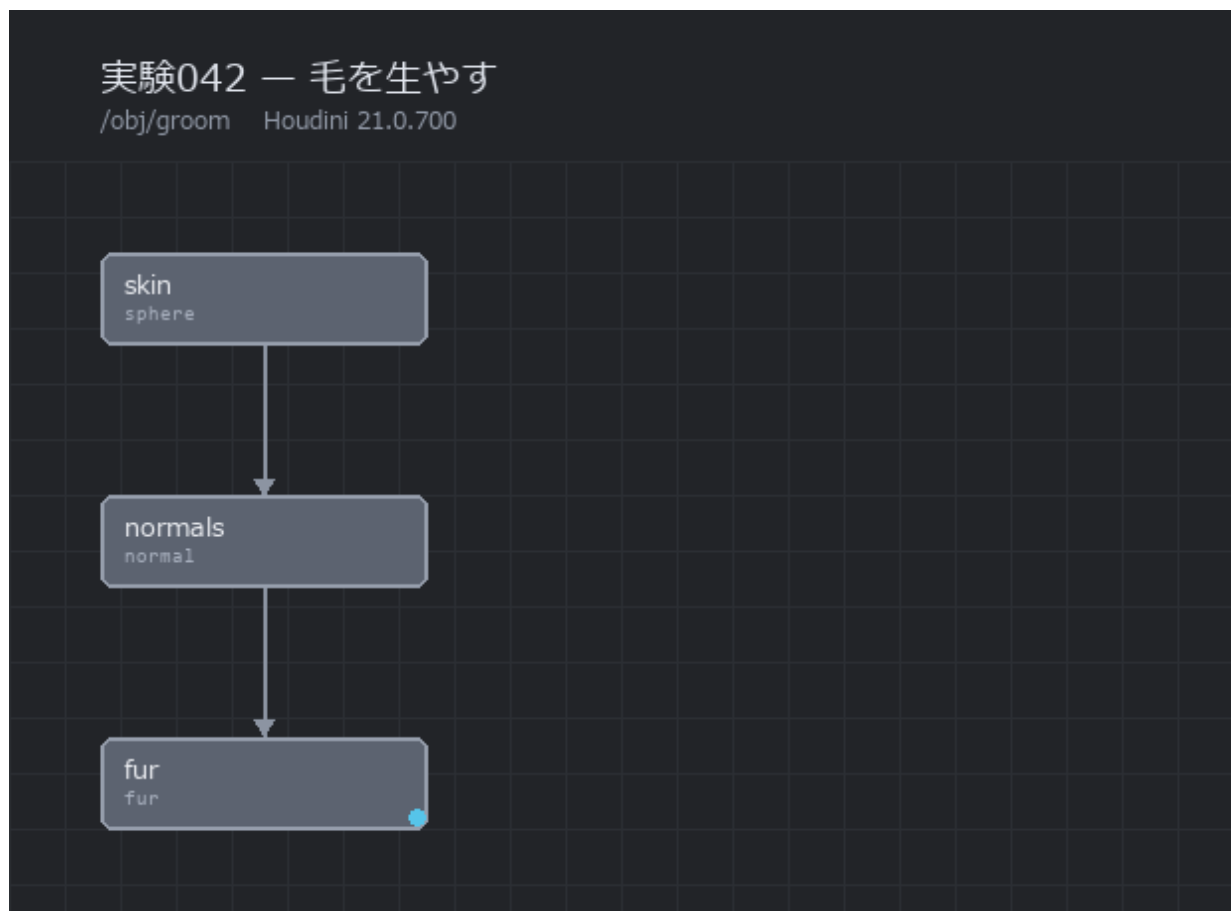
新しい分野に入る。Houdini には毛（グルーム）の仕組みがあり、guide で始まるノードだけで26種ある。ただし最初の1本は fur で足りる。面を渡すと毛が生えてくる。

最初につまずいたのは、どのノードが毛を作るのかだった。名前からすると guideinit が作りそうに見えるが、これは既にあるガイドに風や持ち上がりを付ける整え役で、面だけ渡すと「Not enough sources specified.」で止まる。

数の関係もそのままでは合わない。density に 100 を入れても、生えるのは 1,249本。指定した数そのまま本数になるわけではない。ただし本数を土台の面積で割ると 99.678 で、density の 100 とほぼ一致する。density は「面積1あたりに何本生やすか」だった。

length は純粋な倍率だった。0.125 / 0.25 / 0.5 / 1.0 の4通りで、毛の長さの平均を length で割った値が 0.99808 と小数点以下5桁まで完全に同じ。実験006の height、実験034の amp と同じ性質になる。

ノードの接続

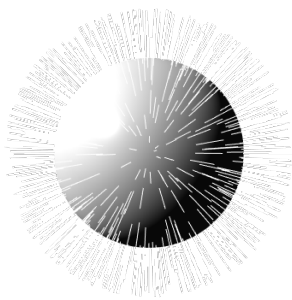


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
fur	fur	(変更なし)

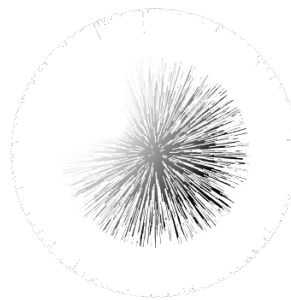
density を変える

土台は 40×40 の球（1,522点 / 1,560面 / 面積 12.53039）。カメラは2枚とも同じ。



Houdini

density 60。毛がまばらで、土台の球が見える。



Houdini

density 600。同じ球とは思えない密度になる。

density	生えた本数	本数 ÷ density	面積あたりの本数	時間
50	612	12.240	48.841	0.01秒
100	1,249	12.490	99.678	0.01秒
200	2,498	12.490	199.355	0.01秒
400	5,050	12.625	403.020	0.01秒

length は純粋な倍率か（実験006・034と同じ問い）

density は 100 のまま、length だけを変えた。毛1本の長さは、区間の長さを足し合わせて測っている。

length	本数	長さの平均	平均 ÷ length	長さのばらつき	毛1本の点数
0.125	1,249	0.12476	0.99808	0.00012	5.00
0.25	1,249	0.24952	0.99808	0.00023	5.00
0.50	1,249	0.49904	0.99808	0.00047	5.00
1.00	1,249	0.99808	0.99808	0.00094	5.00

わかったこと

・ density は本数ではなく「面積あたりの本数」だった。100 を入れて 1,249本。土台の球の面積 12.53039 で割ると 99.678 で、指定した 100 とほぼ一致する。土台を大きくすれば、同じ density でも本数は増える。

・比は4通りでほぼ一定だった。本数÷density が 12.240 / 12.490 / 12.490 / 12.625。ばらつきが残るのは、ばらまきに乱数が入るため。density を2倍にすれば本数も2倍になる。

・length は純粋な倍率だった。4通りで「平均 ÷ length」が 0.99808 と小数点以下5桁まで同じ。実験006の height、実験034の amp と同じ性質になる。Houdini のこの手のパラメータは、倍率として信用してよいことが多い。

・ただし 1.0 ではなく 0.99808 である。指定より 0.19% 短い。4通りすべてで同じ値なので、誤差ではなく決まったずれである。毛が少し反っていて、直線の区間をつないで測ると回り道になるため……とは考えられるが、確かめていないので断定しない（反っているなら逆に長く出るはずで、説明としては弱い）。

・毛1本は5点だった。どの length でも 5点=4区間。長くしても分割は増えない。長い毛をなめらかに曲げたいなら、点数を別に増やす必要がある。

・どのノードが毛を作るのかで、まずつまずいた。名前からすると guideinit が作りそうに見えるが、これは既にあるガイドに風や持ち上げりを付ける整え役で、パラメータも Wind Amount / Lift / Tangential to Skin の11個しかない。面だけ渡すと「Not enough sources specified.」で止まる。

・毛は2段構えで作る。ガイド（案内役、数百本）で形を決め、そこから毛（数万本）に増やす。fur はこの2段を1つにまとめたノードで、最初の1本にはこれでよい。26種のノードは、その2段のそれぞれを細かく調整するためにある。

・面の向き（N）は必要だった。normal を挟んでいる。実験008・030で確かめたとおり、N が無いと向きが決まらない。

次にやること

- ・ガイドを自分で作り、fur ではなく2段構えで組む
- ・hairclump（毛を束ねる）で見え方がどう変わるか測る
- ・043 以降、まだ触っていない分野に進む

毛を束ねる — つなぐ順番が、思っていたのと逆だった

2026-09-24 | Houdini 21.0.700 | /obj/groom | ノード 7 / 接続 9

何を作ったか

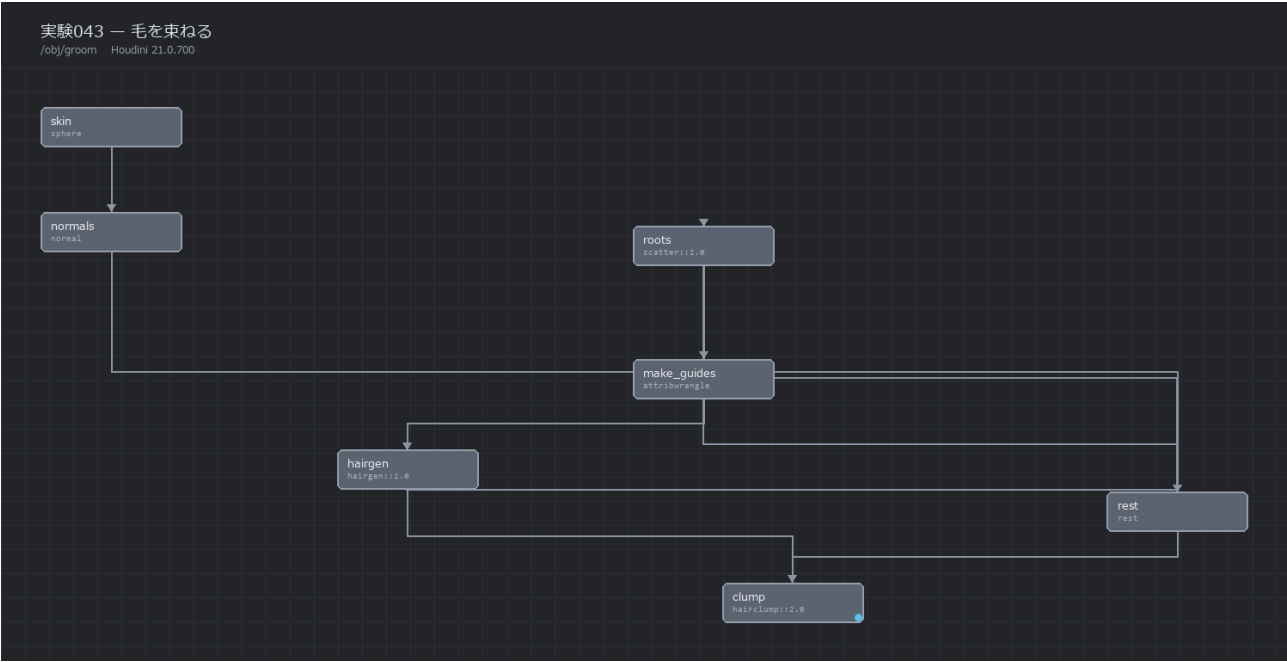
実験042では fur で手早く毛を生やした。今回はその毛を hairclump で束ねる。見た目は、細かい毛が無数に生えた状態から、房にまとまった状態へ変わる。

ところが fur の出力は hairclump につながらない。「Couldn't find skinprim attribute.」で止まる。グルームの道具は「この毛は土台のどの面から生えたか」を番号で知りたがり、fur の出力は N と P しか持っていない。そこでガイドを自分で組み、xyzdist で一番近い面の番号を調べて skinprim に入れた。

それでも動かなかった。原因はつなぐ順番だった。総当たりで確かめたところ、hairgen は「土台 → ガイド」、hairclump は「毛 → 土台」で、2つのノードで順番が逆になる。片方を覚えて、もう片方に当てはめると必ず失敗する。

束ねた結果も測った。clumpsize を 0.05 から 0.8 まで上げると束の数は 9,823 → 27 に減り、毛先どうしの距離は縮む。ただし縮み方は一本調子ではなく、0.2 がいちばん密で、0.8 では逆に広がった。束を大きくすると「太い束」になるためで、これも数字で確かめた。

ノードの接続



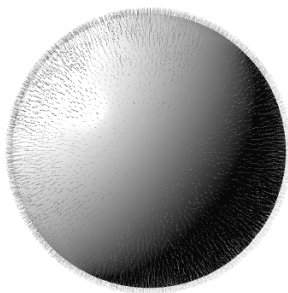
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0
clump	hairclump::2.0	blendremap2pos = 1.0 blendremap2value = 1.0 clumpprofile1interp = 5 clumpprofile1value = 1.0 clumpprofile2interp = 5 clumpprofile2pos = 0.14254385232925415 clumpprofile2value = 0.48605579137802124 clumpprofile3interp = 5 clumpprofile3pos = 0.5679824352264404 clumpprofile3value = 0.11952191591262817 clumpprofile4interp = 5 clumpprofile4pos = 1.0 ...ほか 27 件

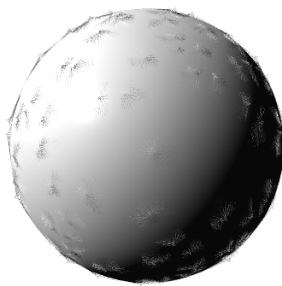
束ねる前と後

土台は 40×40 の球（面積 12.53039）。ガイド300本、density 1000 で 12,516本の毛。カメラは2枚とも同じ。clumpsize 0.4。



Houdini

束ねない。毛が一様に散っていて、全体がうぶ毛のように見える。



Houdini

clumpsize 0.4
で束ねた。144個の房になり、1房あたり約87本。

つないだ順番	ノード	出てきた本数	エラー
土台 → ガイド	hairgen::2.0	12,516	—
ガイド → 土台	hairgen::2.0	出ない	Invalid source .../hairgencore.

つないだ順番	ノード	出てきた本数	エラー
毛 → 土台	hairclump::2.0	12,516	—
土台 → 毛	hairclump::2.0	出ない	Invalid source .../hairclumpcore1.

clumpsize を変えると束の数はどう変わるか

毛は 12,516本で固定。clumpid の種類を数えて束の数とした。

clumpsize	束の数	1束あたりの本数	いちばん大きい束	いちばん小さい束
0.05	9,823	1.27	7	1
0.10	2,607	4.80	22	1
0.20	648	19.31	60	2
0.40	144	86.92	218	14
0.80	27	463.56	966	106

毛先は本当に寄るのか

毛先の点だけを取り出し、いちばん近い毛先までの距離を測った。12,516本から等間隔に1,391本を抜いて比較している（どの条件も同じ本数）。

条件	測った毛先	いちばん近い毛先まで	中央値	束ねない場合との差
束ねない	1,391	0.05885	0.05839	—
clumpsize 0.05	1,391	0.05821	0.05820	-1.1%
clumpsize 0.20	1,391	0.02781	0.02093	-52.7%
clumpsize 0.80	1,391	0.04015	0.03692	-31.8%

なぜ 0.8 では逆に広がったのか

束ごとに、根元の点と毛先の点それぞれについて「束の中心からの平均距離」を測った。これが束の太さにあたる。

clumpsize	束の数	根元の広がり	毛先の広がり	毛先 ÷ 根元
0.05	9,823	0.01381	0.00386	0.279
0.10	2,607	0.02588	0.00627	0.242
0.20	648	0.05408	0.01603	0.296
0.40	144	0.11562	0.05144	0.445
0.80	27	0.26199	0.16565	0.632

わかったこと

- ・つなぐ順番が、2つのノードで逆だった。hairgen::2.0 は「土台 → ガイド」、hairclump::2.0 は「毛 → 土台」。4通りすべてを総当たりで試し、正しい2通りだけが 12,516本を出し、残る2通りは Invalid source で何も出さなかった。片方を覚えて、もう片方に当てはめると必ず失敗する。
- ・fur の出力は、束ねるノードにつながらない。fur が出すのは N と P だけで、skinprim が無い。hairclump は「Couldn't find skinprim

attribute.」で止まる。実験042で使った手軽な道具は、次の工程には渡せない。

- ・土台には rest が要る。球に normal を付けただけでは「Skin input has no rest attribute.」で止まる。rest SOP を1つ挟むと通る。

- ・ガイドは自分で作れた。散らした点から polyline を伸ばし、xyzdist で一番近い面を調べて skinprim と skinprimuv をプリミティブに付ける。これだけで hairgen が受け取る。専用ノードを使わなくても、必要なのは属性2つだった。

- ・束ねて増えるアトリビュートは clumpid ひとつだけだった。束ねる前が id / skinprim / skinprimuv、後が clumpid / id / skinprim / skinprimuv。どの毛がどの束に属するかを示す番号で、これを使えば束ごとに色を変えろといった加工ができる。

- ・clumpsize を上げると束の数は減る。0.05 で 9,823束、0.8 で 27束。0.05 のときは1束あたり 1.27本で、ほとんど束になっていない（毛先の距離も 1.1% しか縮まない）。小さすぎる clumpsize は「効かない」ではなく「1本ずつが1束」になる。

- ・毛先は確かに寄った。ただし一本調子ではない。いちばん近い毛先までの距離は、0.2 で 52.7% 縮む一方、0.8 では 31.8% しか縮まない。clumpsize を上げ続ければ密になる、ではなかった。

- ・その理由は「束が太くなるから」だった。束の毛先の広がり は 0.00386 → 0.16565 と、clumpsize とともに増える。根元に対する比も 0.279 → 0.632 と増えていく。大きい束ほど、根元の広がりに対して毛先が寄りきらない。だから束の中は密でも、毛先どうしの距離の平均は 0.2 を底に反転する。

- ・実験042の density の話が、別の形で裏づけられた。density 1000、土台の面積 12.53039 なので計算上は 12,530本。実際に出たのは 12,516本で、0.1% の差。042 の「density は面積あたりの本数」という結論が、ガイドを自作した今回の組み方でもそのまま成り立つ。

- ・ガイド300本から 12,516本、41.7倍。ガイドの本数は毛の本数に直接は効かない（効くのは density と面積）。ガイドは形を決め、density が数を決める。

次にやること

- ・clumpid を使って束ごとに色を変え、束の境目を目で確かめる
- ・毛にカールやノイズを足す（hairskin 系のノード）
- ・グルームをレンダリングに乗せる（Karma で毛をどう扱うか）

つないだのに、ガイドが1本も使われていなかった — rest が (0,0,0) だった

2026-09-24 | Houdini 21.0.700 | /obj/groom | ノード 8 / 接続 10

何を作ったか

実験043で自作のガイドを hairgen::2.0 につなぎ、12,516本の毛を出した。今回その続きで「ガイドの向きをばらしたら毛はどう変わるか」を測ろうとしたところ、毛先の平均が小数点以下5桁まで1ミリも動かなかった。ガイドを100本にしても900本にしても、毛の本数も束の数もまったく同じ。

つまりガイドは1本も使われていなかった。エラーも警告も出ない。

原因は rest アトリビュートだった。ガイドを作った wrangle では addpoint

で新しい点を作っている。土台から流れてきたジオメトリには rest が付いているので、新しい点にも rest という欄はできるが、中身は (0,0,0) のまま。hairgen は rest

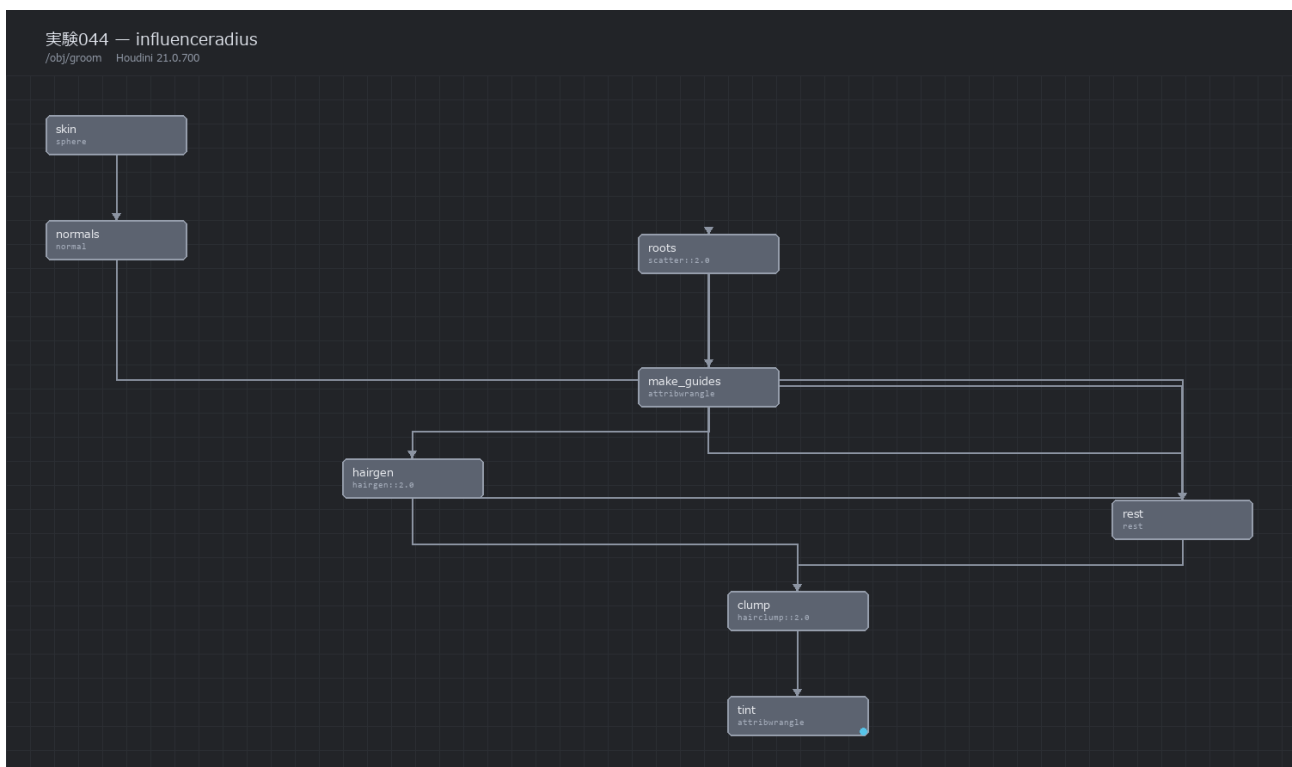
空間で毛とガイドの距離を測るので、ガイドは全部が原点にあることになり、どの毛にも届かなかった。

setpointattrib で rest に実際の位置を入れた瞬間、毛の長さの最大が 0.05000 から 0.49989 に変わった。0.05 は unguidedlength

の既定値で、それまでの毛はすべて「ガイドに拾われなかった毛」の既定の長さだった。

直したうえで influenceradius を上げると、拾われる毛は 18.9% → 73.1% → 100% と増える。既定の 0.05 では2割も拾えない。

ノードの接続



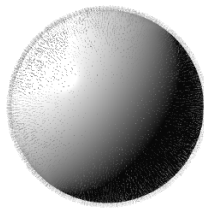
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; float jitter = 0.0; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.4
clump	hairclump::2.0	blendremapramp2pos = 1.0 blendremapramp2value = 1.0 clumpprofile1interp = 5 clumpprofile1value = 1.0 clumpprofile2interp = 5 clumpprofile2pos = 0.14254385232925415 clumpprofile2value = 0.48605579137802124 clumpprofile3interp = 5 clumpprofile3pos = 0.5679824352264404 clumpprofile3value = 0.11952191591262817 clumpprofile4interp = 5 clumpprofile4pos = 1.0 ...ほか 27 件
tint	attribwrangle	class = 1 snippet = int cid = @clumpid; vector c = set(rand(cid * 3 + 0.1), rand(cid * 3 + 1.7), rand(cid * 3 + 2.3)); c = 0.25 + 0.75 *...

rest を直す前と後

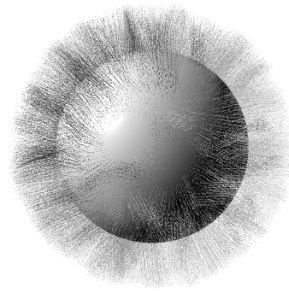
土台は 40×40 の球。ガイド300本（長さ 0.5）、density

1000。カメラは2枚とも同じ。influenceradius 0.2、ガイドの向きはばらしてある。



Houdini

rest が (0,0,0) のまま。ガイドは1本も使われず、長さ 0.05 の短い毛だけが生える。



Houdini

rest を直した。同じつなぎ方・同じパラメータで、ガイドどおりの長さとなり向きになる。

rest	ガイドのばらつき	毛	長さの平均	長さの最大	毛先の広がり
(0,0,0)	0.0	12,516	0.04990	0.05000	0.60504
(0,0,0)	1.2	12,516	0.04990	0.05000	0.60504
直した	0.0	12,516	0.13481	0.49989	0.66185
直した	1.2	12,516	0.13499	0.50000	0.65678

influenceradius を上げると、拾われる毛はどう増えるか

growunguided

を切ると、ガイドに拾われなかった毛は生えない。残った本数が「拾われた毛」になる。長さは growunguided を戻して測った。

influenceradius	全部の毛	拾われた毛	割合	長さの平均	長さの最大
0.05 (既定)	12,516	2,366	18.9%	0.13481	0.49989
0.10	12,516	9,153	73.1%	0.37835	0.49989
0.20	12,516	12,514	100.0%	0.49765	0.49989
0.40	12,516	12,516	100.0%	0.49294	0.49610
0.80	12,516	12,516	100.0%	0.48541	0.48960

clumpid は何を指す番号か

influenceradius 0.20、clumpsize 0.4 で束ねた 12,516本の毛。束は144個。

調べたこと	結果
番号の範囲	204 ~ 12492
0 からの連番か	いいえ (144個が飛び飛び)
番号が毛の id の中にあるか	144 / 144
その id の毛自身が同じ束にいるか	144 / 144
隣の毛が同じ束になる率 (実測)	1,072 / 1,391 = 77.07%

調べたこと	結果
ばらばらに振られていた場合の予想	$1 \div 144 = 0.69\%$

わかったこと

・ガイドは1本も使われていなかった。しかもエラーも警告も出ない。ガイドを100本から900本に増やしても、向きをばらばらに傾けても、毛先の平均が小数点以下5桁まで同一だった。「つないだから効いているはず」は確かめないと言えない。

・原因は rest が (0,0,0) だったこと。wrangle の addpoint で作った点には、既にある属性の欄はできるが中身は既定値のまま入る。土台から rest が流れてきていたので、ガイドの全点の rest が原点になっていた。hairgen は rest 空間で距離を測るので、ガイドは全部が原点にあることになり、どの毛にも届かない。

・気づけたのは、毛の長さが unguidedlength の既定値と一致していたから。長さの最大が 0.05000 ちょうど。ガイドの長さは 0.5 なので、桁がひとつ違う。0.05 は「ガイドに拾われなかった毛」に与えられる既定の長さだった。

・直したら、長さの最大が 0.05000 → 0.49989 に変わった。同時に、ガイドの向きをばらしたときの毛先の差が 0.000000 → 0.005074 になった。ガイドが効いているかどうかは、この2つで判定できる。

・influenceradius の既定 0.05 では、2割弱しか拾えない。ガイド300本を半径1の球に撒くと間隔は約0.2。既定の半径ではほとんどの毛がどのガイドからも遠い。0.20 でようやく 100% (12,514 / 12,516) になる。

・半径を上げすぎると毛が短くなる。長さの平均は 0.49765 (半径0.2) → 0.49294 (0.4) → 0.48541 (0.8)。遠くのガイドまで混ぜると、向きの違うガイドが平均されて縮む。2.5% の差だが、4通りで単調に減っているので誤差ではない。

・clumpid は「束の代表になった毛の番号」だった。0からの連番ではなく 204~12492 の飛び飛びの値で、144個すべてが実在する毛の id と一致し (144/144)、しかもその毛自身が同じ束に属していた (144/144)。番号は通し番号ではなく、束の中心になった毛を指している。

・束はちゃんと空間的にまとまっている。隣の毛が同じ束になる率は 77.07%。ばらばらに振られていたなら $1 \div 144 = 0.69\%$ のはずで、112倍。残りの 23% は束の境目にいる毛になる。

・この実験の測り方そのものが道具になる。「効いているはずのものが効いていない」を見つけるには、入力を大きく変えて出力が動くかを見ればよい。動かなければ、その入力に使われていない。

次にやること

- ・実験043の記事に訂正を出す (あの毛はすべて unguided の毛だった)
- ・ガイドを整える guideprocess / guideadvect を触る
- ・毛をレンダリングに乗せる (Karma で毛をどう扱うか)

土台が動いたとき、毛は付いてくるのか — 入力2をつなぐかどうかだけの差

2026-09-24 | Houdini 21.0.700 | /obj/groom | ノード 8 / 接続 11

何を作ったか

hairgen::2.0 の入力5つある。実験043で読んだラベルは 0:Rest Skin、1:Guides、2:Animated Skin、3:Volumes、4:Guide Interpolation Mesh。つまり「元の形」と「動いている形」を別々に渡す作りになっている。

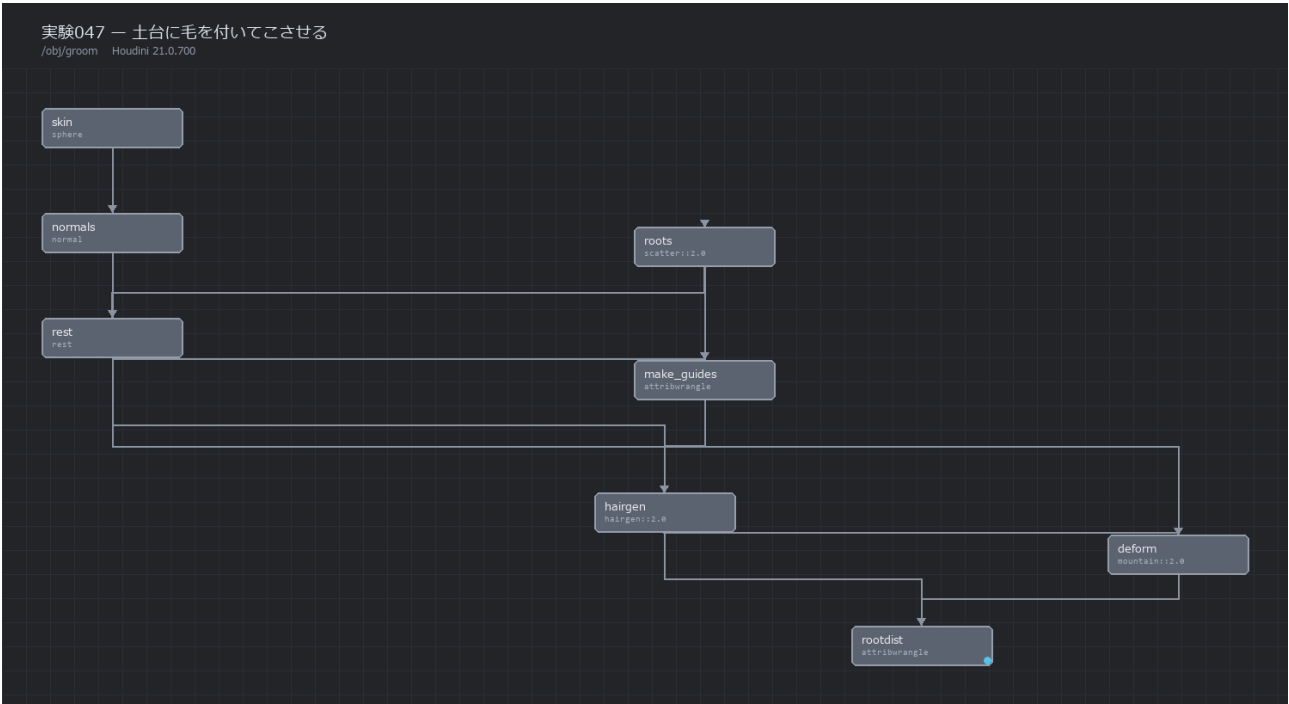
では、動いている形をつながなかったら毛はどうなるのか。土台の球を mountain で少しずつ変形させ、毛の根元から土台の表面までの距離をフレームごとに測った。

つないだ場合、2,500本すべてが全フレームで 0.000000。表面にいる割合は 100.0%。つながらない場合、変形が 0.5 まで進むと平均 0.041807、表面（0.01未満）にいるのは 12.9% だけになる。

そして「離れ」は変形の大きさに比例した。離れ ÷ 変形が 0.08760 / 0.08713 / 0.08629 / 0.08512 / 0.08361。変形の 8~9% しか離れないのは、測っているのが最寄りの表面までの距離だから。近くの別の場所が寄ってくるので、変形そのものの大きさにはならない。

ただし、この差は見た目ではほとんど分からない。0.042 は球の半径の 4% で、毛の長さ 0.5 の 8% にすぎない。目で見つけられないずれは、数字でしか見つからない。

ノードの接続



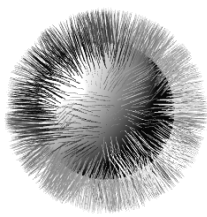
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
rest	rest	(変更なし)
deform	mountain::2.0	elementsiz = 0.9 height = 0.5
roots	scatter::2.0	npts = 200
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	density = 200.0 hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.2 thickness = 0.01
rootdist	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); vector root = point(0, "P", pts[0]); f@rootdist = xyzdist(1, root);

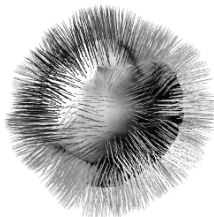
変形の前と後

土台の球を mountain で変形させた。毛は 2,500本、長さ 0.5、太さ 0.010。カメラは2枚とも同じ。OpenGL。



Houdini

フレーム1。変形はまだ 0。



Houdini

フレーム25。変形 0.5。動いている形をつないであるので、根元は表面に乗ったまま。

frame	変形の大きさ	根元の離れ 平均	最大	表面にいる割合
1	0.00000	0.000000	0.000000	100.0%
5	0.08333	0.000000	0.000000	100.0%
10	0.18750	0.000000	0.000000	100.0%
15	0.29167	0.000000	0.000000	100.0%

frame	変形の大きさ	根元の離れ 平均	最大	表面にいる割合
20	0.39583	0.000000	0.000000	100.0%
25	0.50000	0.000000	0.000000	100.0%

動いている形をつながないと、どれだけ離れるか

同じシーンで、hairgen の入力2 (Animated

Skin) だけを外した。毛の根元から、動いている土台の表面までの距離を xyzdist で測っている。

frame	変形の大きさ	根元の離れ 平均	最大	表面にいる割合	離れ ÷ 変形
1	0.00000	0.000000	0.000000	100.0%	—
5	0.08333	0.007300	0.024673	73.6%	0.08760
10	0.18750	0.016337	0.055543	35.4%	0.08713
15	0.29167	0.025169	0.086424	22.8%	0.08629
20	0.39583	0.033692	0.117286	16.3%	0.08512
25	0.50000	0.041807	0.148076	12.9%	0.08361

わかったこと

- ・入力2 (Animated Skin) をつなぐと、根元は完全に表面に乗り続ける。2,500本すべてが全フレームで 0.000000。表面にいる割合は 100.0%。近い、ではなく、ぴったり 0 だった。

- ・つながないと、変形につれて離れていく。変形 0.5 で平均 0.041807、最大 0.148076。表面 (0.01未満) にいるのは 12.9% だけ。

- ・離れは変形の大きさに比例する。離れ ÷ 変形が 0.08760 / 0.08713 / 0.08629 / 0.08512 / 0.08361。5通りで 0.084~0.088

とほぼ一定。わずかに減っていくが、誤差の範囲ではなく単調である。

- ・比が 1 ではなく 0.085 なのは、測っているのが「最寄りの表面までの距離」だから。土台が変形しても、近くの別の場所が寄ってくるので、離れは変形そのものの大きさにはならない。何を測っているかで数字の意味が変わる。

- ・この差は見た目ではほとんど分らない。0.042 は球の半径の 4%、毛の長さ 0.5 の 8% にすぎない。目で見つけられないずれは、数字でしか見つからない。実験044の「ガイドが使われていなかった」も同じ形の問題だった。

- ・「元の形」と「動いている形」を分ける作りの意味。毛の生える位置は元の形の上で決めておき、描くときだけ動いている形に寄せ替える。こうしないとフレームごとに毛の生える場所が変わり、毛がちらつく。

- ・rest は mountain を通しても壊れない。変形ノードは P を動かすだけで rest はそのまま流れる。だから「元の形」と「動いている形」の両方が、同じ rest を持った状態で hairgen に届く。

次にやること

- ・毛を揺らす (Vellum で毛をシミュレーションする)
- ・サンプル数と毛のザラつきの関係 (実験018と同じ問いを毛で)
- ・毛の生える範囲を絵で指定する (density のテクスチャ指定)

APEX に入る —

2,220個の部品と、いちばん小さいグラフ

2026-09-24 | Houdini 21.0.700 | apex::Graph (シーンの中ではなくデータの中にある) | ノード 3 / 接続 2

何を作ったか

APEX は Houdini 20 から入ったリグ (骨組み) の仕組み。特徴は、ノードのネットワークそのものをデータとして持ち歩けること。SOP

のようにシーンの中に置かれるのではなく、ジオメトリの中に入って流れていく。

新しい分野なので、まず大きさを測るところから始めた。apex.Registry().findMatchingNames('*')

で数えると 2,220種。名前空間で分けると geo が 362、sop が 353 で、この2つだけで

715種ある。APEX の中から SOP

の処理をそのまま呼べるということ。名前空間の付いていないもの (算数や比較) が 475種。

次に、いちばん小さいグラフを組んだ。Value を2つ置いて Add につなぐだけ。3 + 4 = 7.0 が出た。

ここで1つつまずいた。Add の2つ目の入力に値を入れても効かない。a=3 / b=4

とパラメータに入れて実行すると、答えは 3.0 になる。エラーは出ない。入力の型を見ると a が Float、b

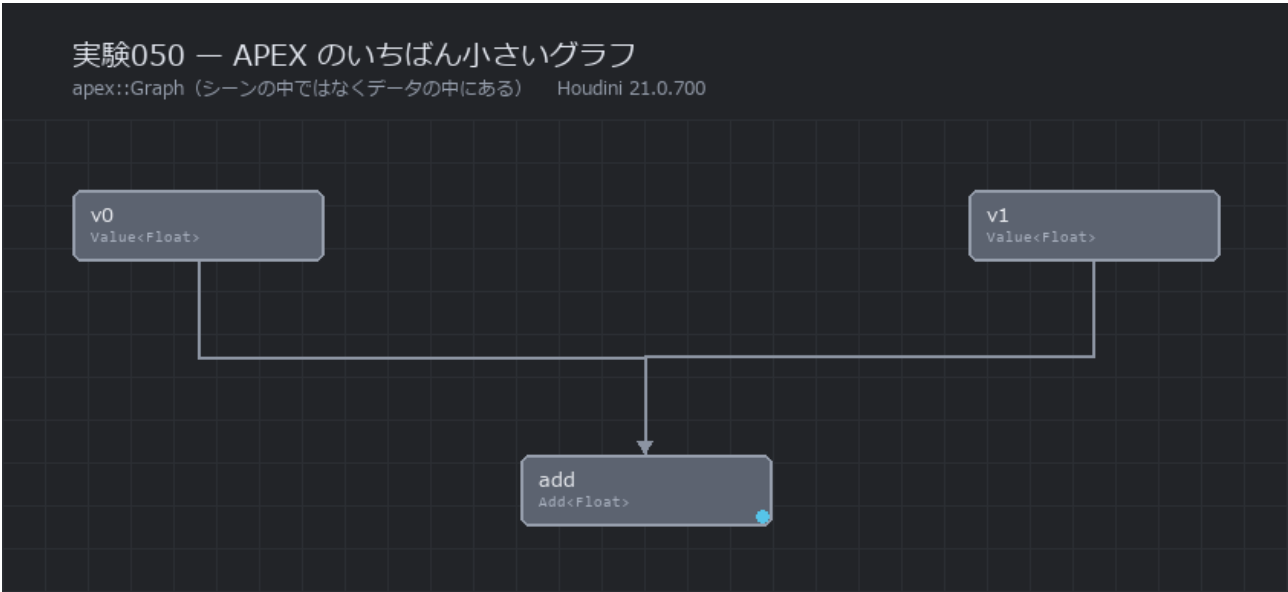
が VariadicArg で、b は「いくつでもつなげる可変長の口」だった。addSubPort

で枝を生やしてからつなぐと動く。可変長なので 1+2+...+10 = 55 もノード1個で通る。

実行は速い。組み立て (compile) が 0.0202ミリ秒、実行が1回あたり

0.499マイクロ秒。組み立ては実行41回分にあたる。

ノードの接続

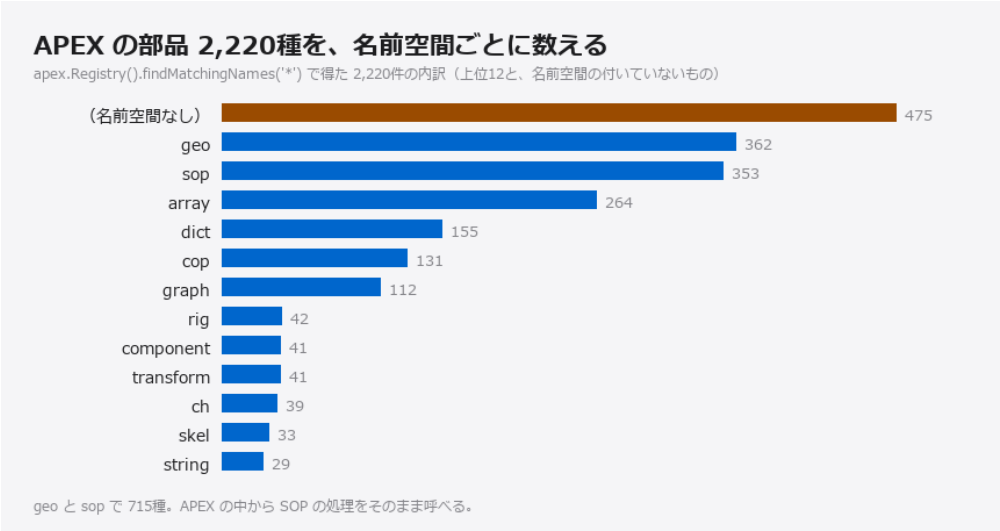


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
v0	Value	parm = 3.0
v1	Value	parm = 4.0
add	Add	a = Float b = VariadicArg

APEX の部品を数える

apex.Registry().findMatchingNames('*') で得た 2,220件を、名前空間で分けた。



上位12と、名前空間の付いていないもの。geo と sop で 715種を占める。

名前空間	数	何のための部品か
(名前空間なし)	475	算数・比較・変換など
geo	362	ジオメトリを触る
sop	353	SOP の処理をそのまま呼ぶ
array	264	配列を扱う
dict	155	辞書を扱う
cop	131	画（Copernicus）を扱う
graph	112	グラフ自身を書き換える
rig	42	リグの部品
component	41	リグの構成要素
transform	41	位置・回転・拡大
ch	39	チャンネル（アニメーション）
skel	33	骨組み
string	29	文字列

いちばん小さいグラフ

Value を2つ置いて Add につなぐ。ノード3個・ポート9個。

調べたこと	結果
ノードの数	3個
ポートの数	9個
1つ目の入力の型	Float
2つ目の入力の型	VariadicArg
3 + 4 の答え	7.0

調べたこと	結果
枝を生やさず parm だけ入れた場合	3.0 (b が無視される)

2つ目の入力に変長

枝（サブポート）を生やした数だけ足せる。ノードは1個のまま。

足す数	式	答え	合っているか
2	$1 + 2$	3.0	はい
3	$1 + 2 + 3$	6.0	はい
5	$1 + 2 + 3 + 4 + 5$	15.0	はい
10	$1 + \dots + 10$	55.0	はい

速さと、SOP 側の入口

実行は10,000回の平均。SOP の種類は `hou.sopNodeTypeCategory()` から数えた。

測ったこと	値
組み立て（compile）	0.0202 ミリ秒
10,000回の実行	0.0050 秒
1回あたりの実行	0.499 マイクロ秒
組み立ては実行何回分か	41回分
apex:: で始まる SOP	38種
kinefx:: で始まる SOP	133種
SOP 全体	1,273種
2つで SOP 全体の	13.4%

わかったこと

- APEX の部品は 2,220種あった。名前空間で分けると geo が 362、sop が 353 で、この2つだけで 715種。APEXの中から SOP の処理をそのまま呼べるということで、リグ専用の小さな言語ではない。

- 名前空間の付いていない部品が 475種。Add / Abs / Clamp / Convert といった算数と変換で、Add<Float> のように型ごとに別の部品になっている（Add だけで16種）。

- いちばん小さいグラフは動いた。Value を2つ置いて Add につなぎ、 $3 + 4 = 7.0$ 。ノード3個・ポート9個。

- 2つ目の入力に値を入れても効かない。 $a=3 / b=4$ をパラメータに入れて実行すると答えは 3.0。パラメータの中身を見ると a も b も入っているのに、b だけ無視される。エラーも警告も出ない。

- 理由は入力の型だった。a が Float、b が VariadicArg<Float>。b は「いくつでもつなげる可変長の口」で、値を直接入れる場所ではない。addSubPort で枝を生やしてからつなぐ。型の合わないつなぎ方をすると Mismatched type: /b:value:Float -> /add:b:VariadicArg<Float> と出るので、そこで気づける。

- 可変長なので、いくつでも足せる。枝を10本生やせば $1+2+\dots+10 = 55.0$ がノード1個で出る。SOP の merge のような考え方が、算数の部品にも入っている。

- ・実行は 0.499マイクロ秒。10,000回で 0.0050秒。組み立て (compile) は 0.0202ミリ秒で、実行41回分。組み立ては1回きり、実行は毎フレーム、という作りに合っている。

- ・ SOP 側の入口は 171種。apex:: が 38種、kinefx:: が 133種で、SOP 全体 1,273種の 13.4%。骨組みまわりだけで SOP の1割を超えている。

- ・ この実験は「新しい分野に入るとき、まず何を測るか」の型でもある。部品の数を知る・いちばん小さいものを動かす・速さを測る。実験042でグループに入ったときも同じ順番だった。

次にやること

- ・ 骨 (skeleton) を作って APEX で動かす
- ・ apex::invokegraph で SOP からグラフを呼ぶ
- ・ リグの部品 (rig 名前空間の42種) を1つずつ見る

骨を作って動かす — rotate と prerotate を取り違えると、関節ごと飛んでいく

2026-09-24 | Houdini 21.0.700 | /obj/rig | ノード 3 / 接続 2

何を作ったか

実験050で APEX の入口を測った。今回はその手前にある骨組み（KineFX）を触る。

KineFX

の骨はただの点と線で持たれている。点1つが関節1つ、線が親子のつながり。名前（name）を付けて kinefx::rigdoctor を通すと、足りない属性が埋まる。手で組んだ時点では P と name

の2つしかなく、rigdoctor が

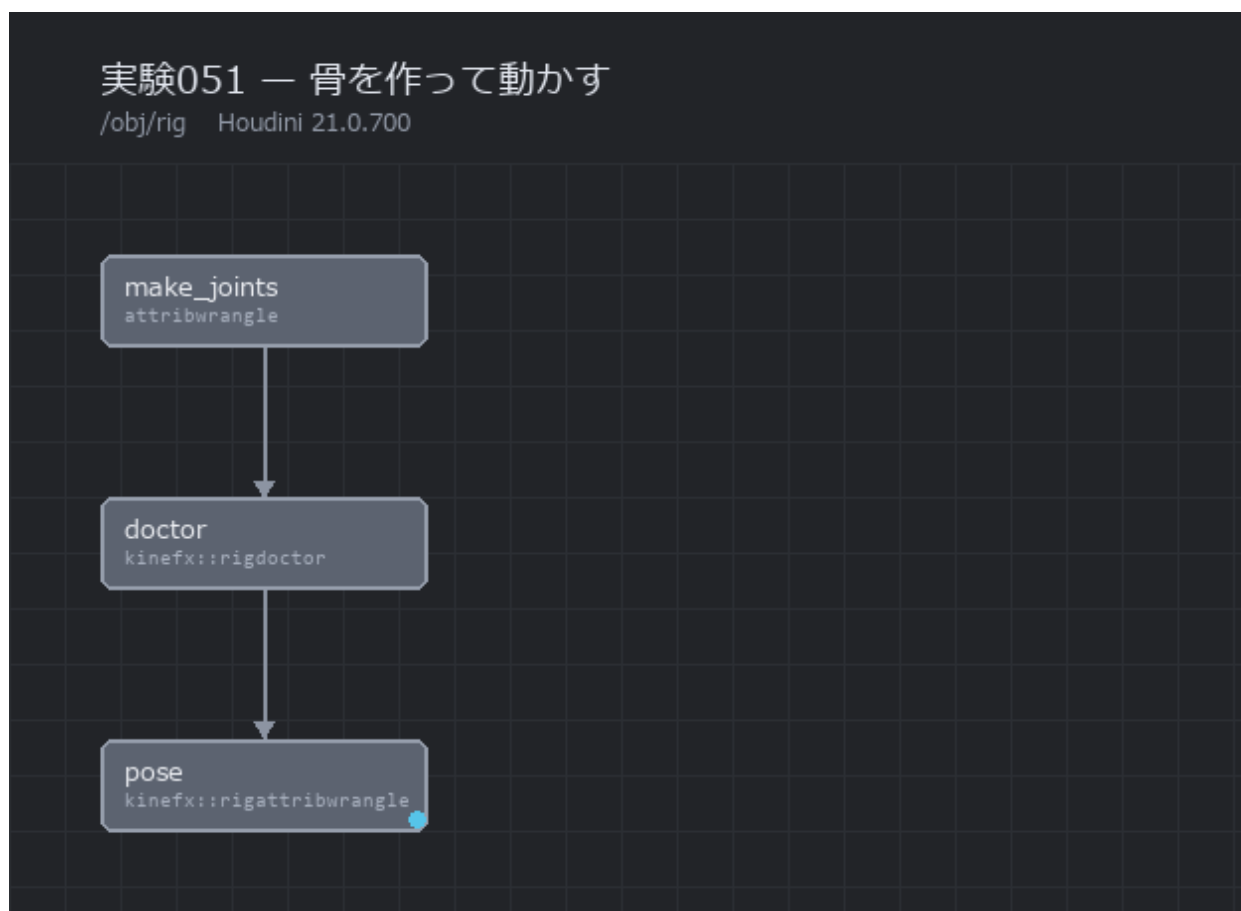
localtransform (4×4の行列) ・ transform (3×3) ・ parent_idx (親が誰か) の3つを足した。

先に式を立てた。根元を原点、関節が (0,1,0) と (0,2,0) にあるなら、真ん中を Z 軸まわりに θ 回したとき先端は $(-\sin \theta, 1 + \cos \theta, 0)$ に来るはず。

6通りの角度で測ったところ、式とのずれは最大 0.0000000060。回した関節そのものの移動は 0.0000000000 だった。

ただし最初は合わなかった。VEX で rotate() を使っていて、90度で先端が (-2, 0, 0) に飛んだ。式の値は (-1, 1, 0) なので、ずれは 1.41。回した関節そのものまで (-1, 0, 0) へ動いていた。原因は掛ける順番で、prerotate() に変えると一致した。

ノードの接続

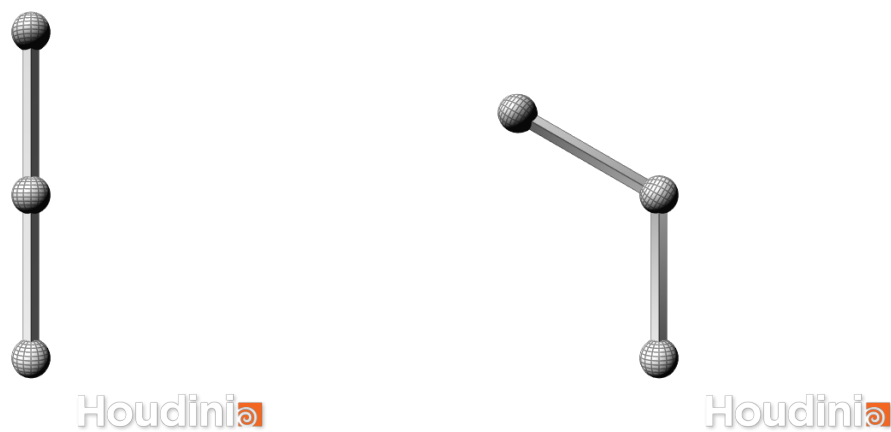


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwran gle	angle = 45.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; // prerotate は「先に回してから動かす」。rotate だと順番が逆になり、 // 関...

曲げる前と後

関節3つの骨。真ん中を Z 軸まわりに 60度回した。線のままでは細くて写らないので polywire
で筒にしてある。カメラは2枚とも同じ。



回す前。関節は (0,0,0) (0,1,0) (0,2,0) に一直線。真ん中を60度。根元と真ん中は動かず、先だけが折れている。

関節	位置	parent_idx	意味
joint0	(0, 0, 0)	-1	親がない=根元
joint1	(0, 1, 0)	0	親は joint0
joint2	(0, 2, 0)	1	親は joint1

真ん中を回すと、先端はどこへ行くか

式は $(-\sin \theta, 1 + \cos \theta, 0)$ 。ずれは実測と式の距離。

角度	先端 実測 x	先端 実測 y	式 x	式 y	ずれ	真ん中の関節の移動
0°	0.000000	2.000000	-0.000000	2.000000	0.000000000	0.000000000
15°	-0.258819	1.965926	-0.258819	1.965926	0.000000014	0.000000000
30°	-0.500000	1.866025	-0.500000	1.866025	0.000000044	0.000000000
45°	-0.707107	1.707107	-0.707107	1.707107	0.000000049	0.000000000

角度	先端 実測 x	先端 実測 y	式 x	式 y	ずれ	真ん中の関節の移動
60°	-0.866025	1.500000	-0.866025	1.500000	0.000000044	0.000000000
90°	-1.000000	1.000000	-1.000000	1.000000	0.000000060	0.000000000

rotate と prerotate を取り違えるとどうなるか

同じ角度・同じ骨で、VEX の1語だけを変えた。

角度	書き方	真ん中 x	真ん中 y	先端 x	先端 y
30°	prerotate (正)	0.000000	1.000000	-0.500000	1.866025
30°	rotate (誤)	-0.500000	0.866025	-1.000000	1.732051
60°	prerotate (正)	0.000000	1.000000	-0.866025	1.500000
60°	rotate (誤)	-0.866025	0.500000	-1.732051	1.000000
90°	prerotate (正)	0.000000	1.000000	-1.000000	1.000000
90°	rotate (誤)	-1.000000	-0.000000	-2.000000	-0.000000

わかったこと

- ・ KineFX の骨は、ただの点と線だった。点1つが関節1つ、線が親子のつながり。普通の wrangle でそのまま組める。専用のノードで作らなくてよい。
- ・ 手で組んだ時点では P と name しか無い。kinefx::rigdoctor が localtransform (4×4) ・ transform (3×3) ・ parent_idx の3つを足す。親子関係は線のつながりから読み取られる（根元の parent_idx は -1）。
- ・ ただし rigdoctor は既定では何も足さない。transformations / inittransforms / outputparentidx のトグルはすべて既定オフ。オンにして初めて属性が出てくる。
- ・ 式とのずれは最大 0.000000060。6通りの角度すべてで、先端は $(-\sin \theta, 1 + \cos \theta, 0)$ にぴたりと来る。回した関節そのものの移動は 0.000000000 で、桁の限界まで動いていない。
- ・ 最初は合わなかった。原因は掛ける順番。VEX の rotate() を使うと、90度で先端が $(-2, 0, 0)$ へ飛ぶ。式の値 $(-1, 1, 0)$ とのずれは 1.41。回した関節そのものまで $(-1, 0, 0)$ へ動いていた。
- ・ prerotate() に変えると一致した。違いは「回してから動かす」か「動かしてから回す」か。関節の位置は移動の分で決まるので、先に回さないと関節ごと原点のまわりを回ってしまう。
- ・ この間違いは絵で見ても気づきにくい。30度なら「少し曲がった骨」に見えてしまう。気づけたのは、先に式を立てて数字で突き合わせたから。90度まで振ると差が 1.41 まで開くので、大きく振ってみるのが見つけるコツ。
- ・ 回転は子にだけ伝わる。真ん中を回しても根元は動かない。親から子へ行列を掛け合わせていく作りなので、自分より上には影響しない。

次にやること

- ・ 骨に肉を付ける (kinefx::jointdeform で皮を付いてこさせる)
- ・ APEX のグラフから骨を動かす (apex::invokekegraph)
- ・ 関節の回る範囲を決める (kinefx::configurejointlimits)

骨に肉を付ける ― 曲げると痩せる。90度で 27% 減った

2026-09-24 | Houdini 21.0.700 | /obj/rigskin | ノード 6 / 接続 7

何を作ったか

実験051で骨を曲げた。今回はその骨に皮（メッシュ）を結び付けて、骨と一緒に動かす。

キャラクターの腕を曲げると肘のあたりが痩せるーというのはこの仕組みでよく言われる話だが、どれくらい痩せるのかは聞いたことがない。測った。

結び付けるノード（`kinefx::jointcaptureproximity`）は、皮に `boneCapture` という属性を1つ足すだけ。これが「この点はどの骨にどれだけ従うか」を持つ。

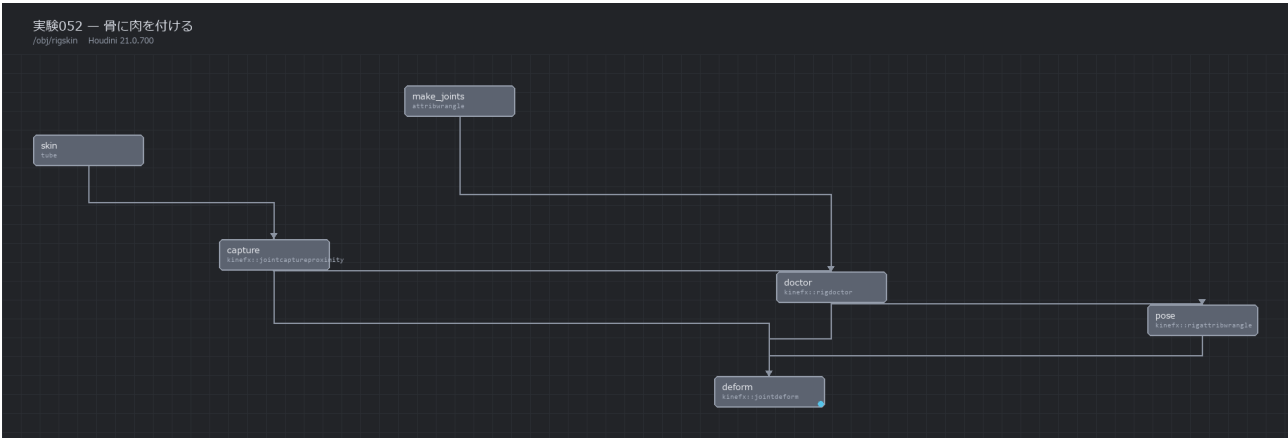
曲げると体積は減った。15度で 0.92%、45度で 7.95%、90度で 27.13%、120度で 40.69%。しかもきれいな式に乗る。減った分を $(1 - \cos \theta)$ で割ると、6通りすべてで 27.1284 になった。つまり 体積の減り = $27.1284\% \times (1 - \cos \theta)$ 。

そして原因が分かった。`maxinfluences`（1つの点が影響を受ける骨の数）が既定で 2 になっている。これを 1 にすると、90度での体積の減りは 27.13% から 0.85% に落ちる。根元の点の移動も 0.443 から 0.0000000000 になる。

ただし代償がある。1本にすると関節が鋭く折れて、内側の角で皮どうしが食い込む。滑らかさと体積は引き換えになっている。

測り方について。体積は面から直接求めた（三角形と原点で四面体を作り、符号付きの体積を足す）。曲げる前の値は 0.636074 で、24角形の筒の理屈の値と小数点以下6桁まで一致した。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	<code>class = 0</code> <code>snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...</code>
doctor	kinefx::rigdoctor	<code>inittransforms = 1</code> <code>outputparentidx = 1</code> <code>transformations = 1</code>

ノード	タイプ	デフォルトから変更したパラメータ
pose	kinefx::rigattribwrange	angle = 90.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotate(m, radians(ch("angle"))), set(0, 0, 1)); ...
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	(変更なし)

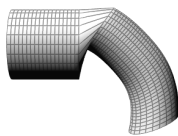
曲げる前と後

皮は 1,440点 / 1,418面 の筒。骨は関節3つ。真ん中を90度回した。カメラは3枚とも同じ。



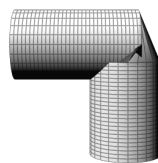
Houdini

曲げる前。体積 0.636074。



Houdini

90度（既定の maxinfluences 2）。なめらかに曲がるが、体積は 27.13% 減る。



Houdini

90度（maxinfluences 1）。体積の減りは 0.85% だが、関節が鋭く折れる。

角度	体積	最初との比	減った分	先端の移動
0°	0.636074	1.000000	0.0000%	0.000000
15°	0.630194	0.990756	0.9244%	0.274093
30°	0.612955	0.963655	3.6345%	0.543495
45°	0.585533	0.920543	7.9457%	0.803599
60°	0.549795	0.864358	13.5642%	1.049952
90°	0.463517	0.728716	27.1284%	1.484857
120°	0.377238	0.593074	40.6926%	1.818571

減り方に式はあるか

減った分を、角度そのもの・その二乗・ $(1 - \cos \theta)$ の3通りで割ってみる。どれかが一定なら、それが式の形になる。

角度	減った分	$\div \theta$	$\div \theta^2$	$\div (1 - \cos \theta)$
15°	0.9244%	0.06163	0.0041084	27.1286

角度	減った分	$\div \theta$	$\div \theta^2$	$\div (1 - \cos \theta)$
30°	3.6345%	0.12115	0.0040384	27.1284
45°	7.9457%	0.17657	0.0039238	27.1284
60°	13.5642%	0.22607	0.0037678	27.1284
90°	27.1284%	0.30143	0.0033492	27.1284
120°	40.6926%	0.33911	0.0028259	27.1284

影響を受ける骨の数を変える（90度で比較）

maxinfluences は「1つの点が、いくつの骨に従うか」。既定は 2。

骨の数	体積	減った分	根元の帯の移動 最大	先端の移動 最大
1	0.630683	0.8475%	0.000000000	1.484857
2（既定）	0.463517	27.1284%	0.442943007	1.484857
3	0.401732	36.8419%	0.468320528	1.316201

どの高さの点が、どれだけ動いたか（90度・既定の2本）

元の高さで8つの帯に分けた。骨は (0,0,0) (0,1,0) (0,2,0)。

元の高さ	点の数	移動の平均	移動の最大
0.00～0.25	192	0.426011	0.442943
0.25～0.50	168	0.387273	0.432379
0.50～0.75	192	0.287112	0.370365
0.75～1.00	168	0.176458	0.268907
1.00～1.25	168	0.188160	0.301651
1.25～1.50	192	0.501344	0.829455
1.50～1.75	168	0.942587	1.125673
1.75～2.01	192	1.286820	1.484857

わかったこと

- ・結び付けても、皮に増える属性は1つだけだった。boneCapture。これが「この点はこの骨にどれだけ従うか」を持つ。骨と皮は別々のジオメトリのまま、この属性1つでつながっている。

- ・曲げると体積は減る。15度で 0.92%、45度で 7.95%、90度で 27.13%、120度で 40.69%。「腕を曲げると肘が痩せる」は、この数字のこと。

- ・減り方には、きれいな式があった。減った分を $(1 - \cos \theta)$ で割ると、6通りすべてで 27.1284（15度だけ 27.1286）。角度そのもので割ると 0.062～0.339、二乗で割っても 0.0041～0.0028 とばらけるのに、この形だけがそろそろ。体積の減り = $27.1284\% \times (1 - \cos \theta)$ 。

- ・この式が正しければ、90度でちょうど 27.1284% になる。 $1 - \cos 90^\circ = 1$ だから。実測もぴたりその値だった。式を立てると、測っていない角度も言える（180度なら 54.26% になるはず）。

- ・原因は maxinfluences が既定で 2 だったこと。1つの点が2本の骨に従うので、関節の近くでは2つの行列が混ざる。混ざった結果は、どちらの行列でもない縮んだ位置になる。

- ・ 1本にすると、体積の減りは 27.13% から 0.85% に落ちる。根元の点の移動も 0.443 から 0.000000000 へ。根元がぴくりとも動かなくなる。
- ・ ただし代償がある。1本にすると関節が鋭く折れて、内側の角で皮どうしが食い込む。なめらかさと体積は引き換え。なお 0.85% という値は、その食い込みの分を含んでいない可能性があるので、そこまでは断定しない。
- ・ 3本にするともっと減る (36.84%)。混ざる行列が増えるほど痩せる。「なめらかにしたいから影響する骨を増やす」は、体積の面では逆効果。
- ・ 既定のままだと、根元の端まで動く。元の高さ 0.00~0.25 の点が平均 0.426 も動いている。真ん中 (0.75~1.00) の 0.176 より大きい。関節から遠いのに大きく動くのは、根元の骨が短く、2本目の骨の重みが残っているため。
- ・ 測り方について。体積は面から直接求めた (三角形と原点で四面体を作り、符号付きの体積を足す)。曲げる前の値 0.636074 は、24角形の筒の理屈の値と小数点以下6桁まで一致した。測り方が正しいことを、先に別の方法で確かめている。

次にやること

- ・ 体積の減りを抑える方法 (dual quaternion skinning) を測る
- ・ 関節の回る範囲を決める (kinex::configurejointlimits)
- ・ APEX のグラフから骨を動かす

痩せない曲げ方はあった — Dual Quaternion で体積の減りが 154分の1 になる

2026-09-24 | Houdini 21.0.700 | /obj/rigskin | ノード 7 / 接続 8

何を作ったか

実験052で、骨を曲げると皮が痩せることを測った。90度で 27.13%、減り方は $27.1284\% \times (1 - \cos \theta)$ というきれいな式に乗っていた。

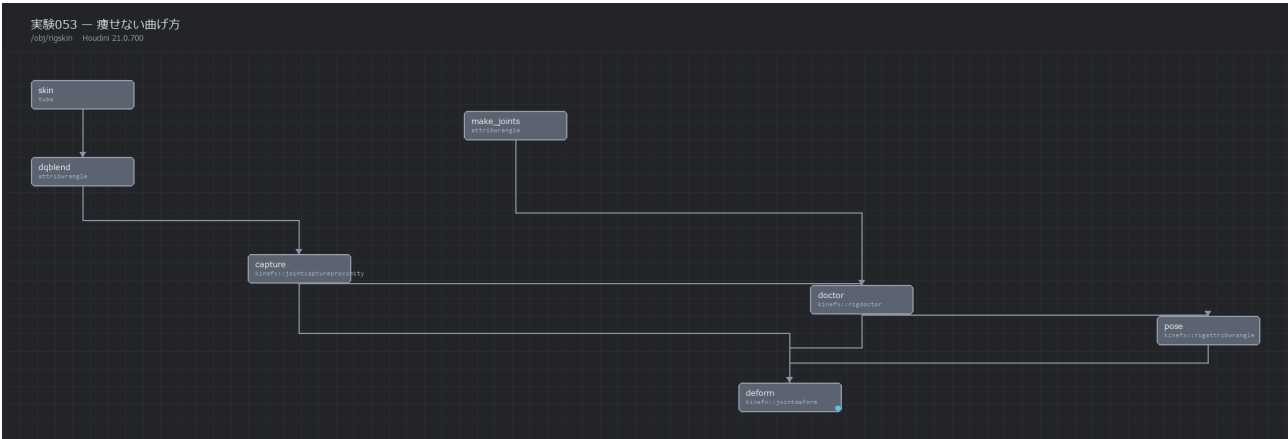
kinefx::jointdeform には混ぜ方が3つある。Linear（既定）、Dual Quaternion、その2つの中間。同じ骨・同じ皮・同じ角度で比べた。

90度での体積の減りは、Linear 27.1284%、Dual Quaternion 0.1764%、中間（半々）14.6904%。Dual Quaternion は Linear の 154分の1 しか痩せない。

そして式のほうも違った。減った分を $(1 - \cos \theta)$ で割ると、Linear は 6通りすべてで 27.1284~27.1286（幅 0.0001）とびたり一定。Dual Quaternion は 0.1513~0.2003（幅 0.0490）でそろわない。中間は 13.6663~15.6573（幅 1.9911）。式に乗るのは Linear だけだった。

「2つの中間」は、そのままでは動かない。点ごとの混ぜ具合を表す属性が要る（Missing dual quaternion blend attribute）。ここでは全点に 0.5 を入れた。すると減りは 14.6904% で、Linear の 27.1284% のほぼ半分。混ぜた割合がそのまま結果に出ている。

ノードの接続



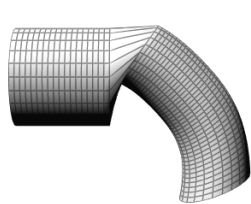
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwran gle	angle = 90.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotate(m, radians(ch("angle"))), set(0, 0, 1)); ...

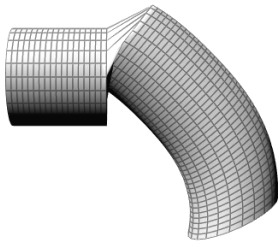
ノード	タイプ	デフォルトから変更したパラメータ
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
dqblend	attribwrangle	snippet = f@dqblend = 0.5;
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	method = 1

同じ90度を、2つのやり方で

皮は 1,440点 / 1,418面 の筒。骨は関節3つ。カメラは2枚とも同じ。



Houdini



Houdini

Linear（既定）。内側が食い込み、体積は 27.13% 減る。Dual Quaternion。太さが保たれ、体積の減りは 0.18%。

角度	Linear（既定）	Dual Quaternion	2つの中間
0°	0.0000%	0.0000%	0.0000%
15°	0.9244%	0.0052%	0.4657%
30°	3.6345%	0.0205%	1.8417%
45°	7.9457%	0.0459%	4.0666%
60°	13.5642%	0.0808%	7.0413%
90°	27.1284%	0.1764%	14.6904%
120°	40.6926%	0.3005%	23.4860%

それぞれ式に乗るか（減った分 ÷ (1 - cos θ))

一定なら、その形の式に乗っている。

角度	Linear（既定）	Dual Quaternion	2つの中間
15°	27.1286	0.1513	13.6663

角度	Linear（既定）	Dual Quaternion	2つの中間
30°	27.1284	0.1532	13.7469
45°	27.1284	0.1568	13.8844
60°	27.1284	0.1616	14.0827
90°	27.1284	0.1764	14.6904
120°	27.1284	0.2003	15.6573
ばらつきの幅	0.0001	0.0490	1.9911

中心からの太さ（90度するとき、高さの帯ごと）

その帯の点が、帯の中心からどれだけ離れているかの平均。曲げる前の筒の半径は 0.32。

元の高さ	Linear（既定）	Dual Quaternion	2つの中間
0.00～0.50	0.305559	0.353538	0.329018
0.50～0.75	0.281015	0.329243	0.304530
0.75～1.00	0.279344	0.327047	0.302601
1.00～1.25	0.279592	0.327048	0.302731
1.25～1.50	0.328019	0.341245	0.333388
1.50～2.01	0.350916	0.350916	0.350916

わかったこと

- ・ Dual Quaternion は 154分の1 しか痩せない。90度で Linear が 27.1284% 減るのに対し、0.1764%。 $27.1284 \div 0.1764 = 153.8$ 倍の差。
- ・ 式に乗るのは Linear だけだった。(1 - cos θ) で割った値の幅は、Linear が 0.0001、Dual Quaternion が 0.0490、中間が 1.9911。Linear は式で言い切れるが、Dual Quaternion はこの形では表せない。
- ・ Dual Quaternion も、角度を上げると少しずつ痩せる。0.0052%（15度）→ 0.3005%（120度）。ゼロではない。ただし 120度でも 0.3% なので、実用上は無視できる。
- ・ 「2つの中間」は、そのままでは動かない。「Missing dual quaternion blend attribute」で止まる。点ごとの混ぜ具合を表す属性を、自分で作って名前を指定する。ここでは全点に 0.5 を入れた。
- ・ 半々に混ぜると、減りもほぼ半分になる。14.6904% は Linear の 27.1284% の 0.5415倍。混ぜた割合がそのまま結果に出る。部位ごとに混ぜ方を変えられるのが、この方式の使いどころになる。
- ・ 太さで見ても差ははっきり出る。関節の近く（高さ 0.75～1.00）で、Linear が 0.279344、Dual Quaternion が 0.327047。元の半径 0.32 に対して、Linear は 13% 細く、Dual Quaternion はほぼそのまま。
- ・ 関節から離れた場所では3つとも同じ。高さ 1.50～2.01 の帯はどのやり方でも 0.350916 で完全に一致する。骨1本にしか従っていない場所では、混ぜ方は関係ない。この一致は、測り方が正しいことの確認にもなっている。
- ・ ではなぜ Linear が既定なのか。この実験からは分からない。速さか、ねじれの扱いか、互換性か。確かめていないので断定しない。次に測るとすれば、そこになる。

次にやること

- Linear と Dual Quaternion の速さを比べる
- ねじれ（ひねり）を入れたときの違いを測る
- 関節の回る範囲を決める（`kinefx::configurejointlimits`）

なぜ Linear が既定なのか — 速さではなかった。差は 4%

2026-09-24 | Houdini 21.0.700 | /obj/rigskin | ノード 6 / 接続 7

何を作ったか

実験053で、Dual Quaternion のほうが 154倍も痩せないと分かった。ではなぜ Linear が既定のままなのか。考えられるのは2つ。遅いのか、ねじれで挙動が違うのか。

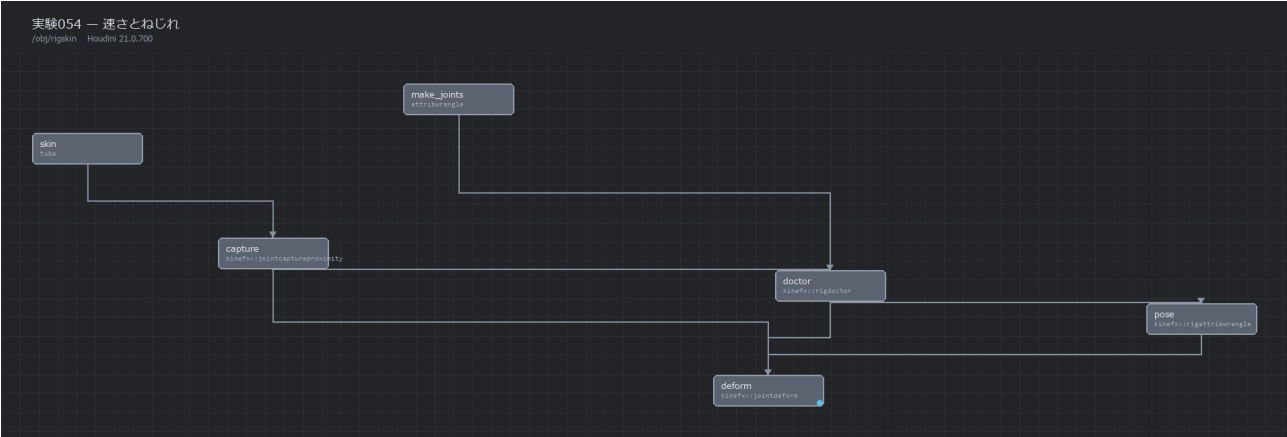
速さから測った。実験009の教訓を使い、毎回新しいノードを作って初回のクックを測る。414,720点の皮で Linear 14.66ミリ秒、Dual Quaternion 15.21ミリ秒。倍率 1.04。点1つあたりの差は約 1.8ナノ秒。5通りの大きさを倍率は 0.98~1.12 に収まる。速さは理由にならない。

ねじれでも Linear は不利だった。骨の軸まわりに 179度ひねると、Linear は体積が 66.81% 減る。有名な「飴の包み紙」の形になり、真ん中がほぼ潰れる。Dual Quaternion は 0.24% しか減らない。

ただし Dual Quaternion にも癖があった。ねじれが根元まで広がる。170度を指定したとき、根元の帯（高さ 0~0.5）が Linear では 14.1度しかねじれないのに、Dual Quaternion では 58.9度ねじれる。そのうえ、高さ 1.10~1.50 の帯が 99.2度で止まっているのに、いちばん上の帯は 170度。途中に段差ができる。絵にもはっきり出る。

つまり「なぜ Linear が既定なのか」は、この2つの実験では決まらなかった。速さでもなく、ねじれの体積でもない。分かったのは、Dual Quaternion が別の癖を持っていることだけである。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwran gle	angle = 170.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotate(m, radians(ch("angle"))), set(0, 1, 0)); ...

ノード	タイプ	デフォルトから変更したパラメータ
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	method = 1

骨の軸まわりに 170度ひねる

皮は 1,440点の筒。骨は関節3つ。カメラは2枚とも同じ。



Houdini



Houdini

Linear。真ん中がほぼ潰れる。179度で体積は 66.81% 減る。

Dual Quaternion。太さは保たれるが、途中に段差ができる。

ひねり	Linear	減った分	Dual Quaternion	減った分
0°	0.636074	0.0000%	0.636074	0.0000%
45°	0.574068	9.7482%	0.636140	-0.0105%
90°	0.423900	33.3568%	0.635838	0.0371%
135°	0.273535	56.9964%	0.635251	0.1294%
170°	0.214342	66.3024%	0.634689	0.2177%
179°	0.211093	66.8131%	0.634541	0.2409%

速さ（毎回新しいノードを作って、初回のクックを測る）

結び付け（capture）までは先に計算を済ませてあるので、測っているのは皮を動かす処理だけ。1回目は準備の分が乗るので捨てている。

点の数	面の数	Linear	Dual Quaternion	倍率	1点あたりの差
2,880	2,858	4.47 ms	4.36 ms	0.98	-36.15 ns

点の数	面の数	Linear	Dual Quaternion	倍率	1点あたりの差
11,520	11,474	4.81 ms	5.41 ms	1.12	51.68 ns
46,080	45,986	5.75 ms	5.76 ms	1.00	0.20 ns
184,320	184,130	9.06 ms	9.78 ms	1.08	3.88 ns
414,720	414,434	14.66 ms	15.21 ms	1.04	1.32 ns

高さごとに、実際は何度ねじれたか（170度を指定）

点の x-z 平面での角度が、元の位置から何度動いたかの中央値。軸の近くは角度が不安定なので、外側だけを使っている。

元の高さ	Linear	Dual Quaternion
0.00～0.50	14.1°	58.9°
0.50～0.90	59.7°	80.6°
0.90～1.10	102.6°	87.9°
1.10～1.50	142.2°	99.2°
1.50～2.01	170.0°	170.0°

180度を越えるようになるか

Dual Quaternion

は角度が半回転を越えると近道を通る、と言われる。体積で見ると限りそうはならなかった。

ひねり	Linear 減った分	Dual Quaternion 減った分
170°	66.3024%	0.2177%
179°	66.8131%	0.2409%
181°	66.8149%	0.5458%
200°	64.8223%	0.4665%
270°	33.4623%	0.1981%
350°	0.5167%	0.0107%

わかったこと

- ・速さは理由ではなかった。414,720点で Linear 14.66ミリ秒、Dual Quaternion 15.21ミリ秒。倍率 1.04。5通りの大きさが 0.98～1.12 に収まる。点1つあたりの差は約 1.8ナノ秒。
- ・ねじれでも Linear は不利だった。179度で体積が 66.81% 減る。真ん中がほぼ潰れて、有名な「飴の包み紙」の形になる。Dual Quaternion は 0.24%。
- ・ただし Dual Quaternion にも癖がある。ねじれが根元まで広がる。170度を指定したとき、根元の帯（高さ 0～0.5）は Linear で 14.1度しかねじれないのに、Dual Quaternion では 58.9度。動かしていない側まで巻き込む。
- ・そのうえ途中に段差ができる。高さ 1.10～1.50 の帯が 99.2度で止まっているのに、いちばん上の帯は 170度。その境目が絵でも見える。Linear は 14 → 60 → 103 → 142 → 170 と、なめらかにつながっている。

- ・ 180度を越えても、体積では破綻しなかった。Dual Quaternion は半回転を越えると近道を通る、と言われる。181度で 0.55%、270度で 0.20%、350度で 0.01%。少なくとも体積で見るかぎり、そうはならない。向きまでは調べていないので断定はしない。

- ・ Linear は 350度で元に戻る。減りが 0.52% まで下がる。一周に近づけば、ねじれていないのと同じだから。66.8% でいちばん潰れるのは 179度 = 半回転のところ。

- ・ 結局「なぜ Linear が既定なのか」は決まらなかった。速さでもなく、ねじれの体積でもない。分かったのは、Dual Quaternion が「ねじれを局所に留めない」という別の癖を持つことだけ。

- ・ 分からなかったことを、分からないまま残す。ここで「互換性のためだろう」と書くのは簡単だが、測っていない。この実験で言えるのは、速さとねじれの体積という2つの筋が消えた、ということ。

- ・ 測り方について。結び付け (capture) までは先に計算を済ませてから測っている。そうしないと、両方に共通の重い処理が時間に混ざる。比べたいものだけを測る形に作り替える。

次にやること

- ・ ねじれの向きまで調べて、180度越えの挙動を確かめる
- ・ 関節の回る範囲を決める (kinexfx::configurejointlimits)
- ・ APEX のグラフから骨を動かす

逆運動学（IK） — 届かない場所を指すと、伸びきってそこで止まる

2026-09-24 | Houdini 21.0.700 | apex::Graph (rig::TwoBoneIK) | ノード 5 / 接続 4

何を作ったか

ここまでは関節を1つずつ回して形を作ってきた（順運動学）。IK
はその逆で、先端を「ここに来い」と指定すると、途中の関節が勝手に決まる。

使ったのは APEX の部品 rig::TwoBoneIK。入力は 4×4 の行列で root / mid / tip /
goal、出力は解けた3つの行列。実験050で組んだグラフの作り方がそのまま使える。

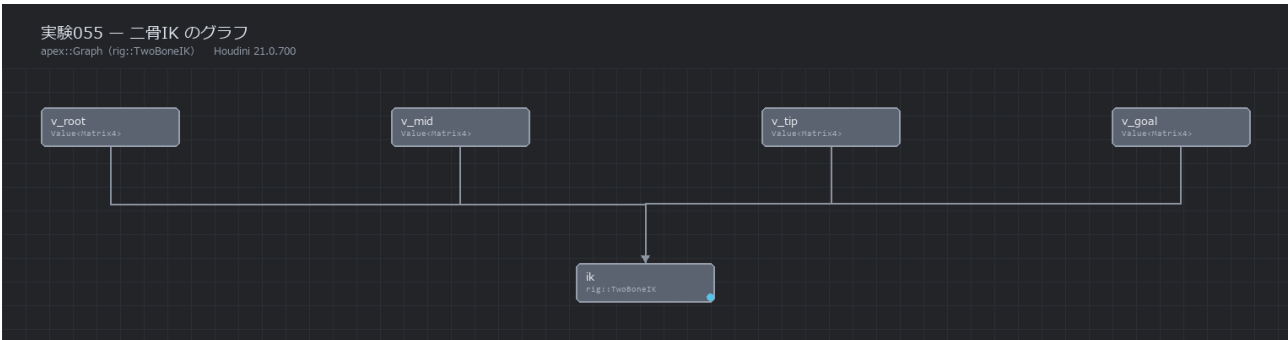
骨は (0,0,0) → (0,1,0) → (0,2,0) の3関節。長さは 1 と 1 なので、根元から先端までは最大でも 2
までしか届かない。先に式を立てた。目標までの距離を d とすると、 $d \leq 2$ なら誤差 0、 $d > 2$ なら誤差
 $= d - 2$ 。

9通りの距離で測ったところ、式とのずれは 0.000000000。すべての桁で一致した。骨の長さも
1.000000 のまま動かない（幅 0.000000000）。

stretch を入れると、届かない距離でも目標に届く。そのかわり骨が伸びる。伸びる比は 2.5 で
1.25、3.0 で 1.5、4.0 で 2.0。つまり「目標までの距離 ÷ 届く距離」そのもの。

意外だったのは blend。0 から 1
へ動かすと、先端は目標へまっすぐ近づくのだと思っていた。実際には途中でいったん遠ざかり（1.4168
→ 1.4627）、しかも z 方向へ 0.858 も膨らんでから戻ってくる。blend
は「目標へ近づく割合」ではない。

ノードの接続

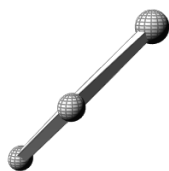


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
v_root	Value	parm = (0, 0, 0)
v_mid	Value	parm = (0, 1, 0)
v_tip	Value	parm = (0, 2, 0)
v_goal	Value	parm = 目標の位置
ik	rig::TwoBoneIK	blend = 1.0 stretch = False

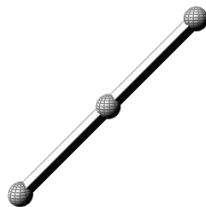
届く場合と、届かない場合

骨は関節3つ、長さ 1 と 1。大きい球が目標。カメラは2枚とも同じ。



Houdini

目標まで 1.5。届くので、肘を曲げてぴたりと合わせる。



Houdini

目標まで 4.0。届かないので、まっすぐ伸びきってそこで止まる。

目標まで	先端の位置	誤差 実測	誤差 予測	差	骨1	骨2
0.50	(0.35355, 0.35355, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
1.00	(0.70711, 0.70711, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
1.50	(1.06066, 1.06066, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
1.90	(1.34350, 1.34350, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
2.00	(1.41421, 1.41421, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
2.10	(1.41421, 1.41421, 0.00000)	0.1000000	0.1000000	0.000000000	1.000000	1.000000
2.50	(1.41421, 1.41421, 0.00000)	0.5000000	0.5000000	0.000000000	1.000000	1.000000
3.00	(1.41421, 1.41421, 0.00000)	1.0000000	1.0000000	0.000000000	1.000000	1.000000
4.00	(1.41421, 1.41421, 0.00000)	2.0000000	2.0000000	0.000000000	1.000000	1.000000

stretch を入れる

届かない距離でも目標に届くようになる。そのかわり骨が伸びる。

目標まで	誤差	骨1	骨2	合計 ÷ 2	目標 ÷ 届く距離
1.00	0.000000000	1.000000	1.000000	1.000000	0.50
2.00	0.000000000	1.000000	1.000000	1.000000	1.00
2.50	0.000000121	1.250000	1.250000	1.250000	1.25
3.00	0.000000000	1.500000	1.500000	1.500000	1.50
4.00	0.000000194	2.000000	2.000000	2.000000	2.00

blend を 0 から 1 へ動かす（目標まで 1.5）

0 なら元の姿勢、1 なら解いた姿勢。その間はどう動くのか。

blend	先端の位置	元からの移動	目標までの残り
0.00	(0.00000, 2.00000, 0.00000)	0.000000	1.416813
0.25	(0.10741, 1.76356, 0.85836)	0.896788	1.462715
0.50	(0.72227, 1.30375, 1.11368)	1.498905	1.189069
0.75	(1.19560, 1.02337, 0.64122)	1.671652	0.656324
1.00	(1.06066, 1.06066, 0.00000)	1.416813	0.000000

わかったこと

- ・式と完全に一致した。9通りの距離すべてで、実測の誤差と予測の誤差の差が 0.000000000。届く範囲では誤差ゼロ、届かなければ $d - 2$ 。
- ・届かないとき、先端は (1.41421, 1.41421, 0) で止まる。これは原点から距離 2.0 の点。目標を 2.1 にしても 4.0 にしても、まったく同じ場所で止まる。伸びきった腕は、それ以上どうにもならない。
- ・骨の長さは 1.000000 のまま。9通り × 2本 = 18個の値の幅が 0.000000000。IK は関節を回すだけで、骨を伸ばさない。
- ・stretch を入れると、届かない距離でも届く。誤差は 2×10^{-7} 以下。そのかわり骨が伸びる。
- ・伸びる比にも式があった。2.5 で 1.25、3.0 で 1.5、4.0 で 2.0。「目標までの距離 ÷ 届く距離」そのもの。2本とも同じだけ伸びる。
- ・blend は「目標へ近づく割合」ではなかった。0 から 1 へ動かすと、目標までの残りは 1.4168 → 1.4627 → 1.1891 → 0.6563 → 0。途中でいったん遠ざかる。
- ・しかも平面から外れる。元の骨も目標も $z = 0$ の面にあるのに、blend 0.25 では先端が $z = 0.858$ ままで膨らむ。行列そのものを混せているので、途中の姿勢は「途中の形」であって「途中の位置」ではない。
- ・APEX の部品は、こういう確かめ方に向いている。ジオメトリを介さず、行列を入れて行列を受け取るだけ。シーンを組まずに、数字だけで挙動を確かめられる。
- ・なお SOP の kinexfx::ikchains::2.0 も試したが、エラーも警告も出ないまま何も起きなかった。goal の指定を6通り、入力を作り方を4通り試しても骨は動かない。原因は分かっていないので断定しない。APEX の部品のほうは素直に動いた。

次にやること

- ・ 3関節より長い鎖の IK (FABRIK) を測る
- ・ APEX のグラフをジオメトリに埋め込んで持ち歩く
- ・ 関節の回る範囲を決める

長い鎖の IK — 式は同じ。ただし解き方は2種類あり、得意な範囲が逆

2026-09-24

何を作ったか

実験055では関節3つ（骨2本）の IK を測り、届く距離までは誤差ゼロ、越えたら $d - 2$ という式にぴたり乗った。関節が増えたらどうなるか。骨の長さが1のまま n 本つながっているなら、届く距離は素直に n のはずだ。

関節を 3 / 5 / 9 / 17 と変え、届く距離の 0.3倍から 1.5倍まで目標を置いて測った。届かない側では、式とのずれが 0.000000000。17関節・目標24でも、先端はきっちり距離16の位置で止まる。骨の長さは全24通りで幅 0.000000000 だった。

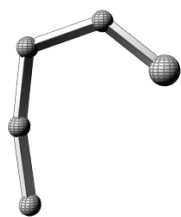
ただし届く側は誤差ゼロではない。IK は繰り返して近づける解き方なので、わずかに残る。しかもその残りは鎖が長いほど大きい。目標をちょうど届く距離に置いたとき、骨2本で 0.000076、16本で 0.002775、32本で 0.006901。

そして solver というパラメータがある。0（既定） / 1 / 2 / 3 のうち、2 と 3 は 0 とまったく同じ結果になった。1 だけが違う。

面白いのは、どちらが正確かが条件でひっくり返ること。関節9個で測ると、届く距離の8割までは solver=1 が 5〜100倍正確。9割を越えると逆転して、今度は既定のほうが 5〜58倍正確になる。

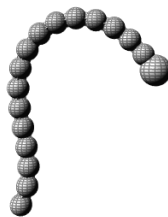
関節の数を変える

目標は届く距離の 0.3 / 0.7 / 0.95 / 1.0 / 1.05 / 1.5 倍。誤差の予測は $\max(0, \text{目標までの距離} - \text{骨の本数})$ 。



Houdini

関節5個（骨4本）。目標は届く距離の6割。



Houdini

関節17個（骨16本）。同じ割合でも、長い鎖はゆるく巻く。

関節	骨	届く距離	目標まで	誤差 実測	誤差 予測	差	骨の長さの幅
3	2	2.0	1.900	0.0001076	0.0000000	0.000107637	0.000000000
3	2	2.0	2.000	0.0000762	0.0000000	0.000076154	0.000000000

関節	骨	届く距離	目標まで	誤差 実測	誤差 予測	差	骨の長さの幅
3	2	2.0	2.100	0.1000000	0.1000000	0.00000000 0	0.00000000 0
5	4	4.0	4.000	0.0007615	0.0000000	0.00076150 4	0.00000000 0
5	4	4.0	6.000	2.0000000	2.0000000	0.00000000 0	0.00000000 0
9	8	8.0	8.000	0.0014441	0.0000000	0.00144411 8	0.00000000 0
9	8	8.0	12.000	4.0000000	4.0000000	0.00000000 0	0.00000000 0
17	16	16.0	16.000	0.0027750	0.0000000	0.00277503 2	0.00000000 0
17	16	16.0	24.000	8.0000000	8.0000000	0.00000000 0	0.00000000 0

届かないとき、先端はどこで止まるか（関節9個・骨8本）

根元から先端までの距離。骨の本数は 8。

目標まで	先端までの距離	骨の本数との差
8.0	7.998555882	-0.001444118
9.0	8.000000000	0.000000000
12.0	8.000000000	0.000000000
20.0	8.000000000	0.000000000

鎖が長いほど、残る誤差は大きい（目標＝届く距離ちょうど）

いちばん厳しい条件。伸びきる直前で、繰り返しの残りがいちばん出る。

関節	骨	既定(0)の誤差	solver=1 の誤差
3	2	0.000076154	0.019919474
5	4	0.000761504	0.044688692
9	8	0.001444118	0.083483171
17	16	0.002775032	0.145489437
33	32	0.006900571	0.252714115

2つの解き方は、得意な範囲が逆（関節9個・骨8本）

遠さの割合＝目標までの距離 ÷ 届く距離。

遠さの割合	目標まで	既定(0)の誤差	solver=1 の誤差	良いほう
0.10	0.800	0.000003436	0.000000652	solver=1
0.30	2.400	0.000151835	0.000004945	solver=1
0.50	4.000	0.000024090	0.000004584	solver=1
0.70	5.600	0.000286937	0.000014048	solver=1

遠さの割合	目標まで	既定(0)の誤差	solver=1 の誤差	良いほう
0.80	6.400	0.000369785	0.000140004	solver=1
0.90	7.200	0.000261699	0.001405246	既定
0.95	7.600	0.000406530	0.004909842	既定
0.99	7.920	0.000552590	0.044165596	既定
1.00	8.000	0.001444118	0.083483171	既定

わかったこと

・式は関節が増えても同じだった。届かない側では、実測と予測の差が 0.000000000。17関節で目標を24に置いても、先端はきっちり距離16の位置で止まる。届く距離=骨の本数×長さ。

・骨の長さは全24通りで幅 0.000000000。関節が16本つながっていても、1本も伸び縮みしない。実験055の「IK は関節を回すだけ」が、長い鎖でもそのまま成り立つ。

・ただし届く側の誤差はゼロではない。IK は繰り返して近づける解き方なので、わずかに残る。関節3つの二骨IK（実験055）が全桁一致だったのとは違う。あちらは式で一発に解けるが、こちらは近づけていく。

・残る誤差は、鎖が長いほど大きい。目標をちょうど届く距離に置いたとき、骨2本で 0.000076、16本で 0.002775、32本で 0.006901。骨の本数を2倍にすると、誤差もおおよそ2倍。

・いちばん誤差が出るのは「伸びきる直前」。関節9個で見ると、遠さ 0.5 では 0.000024なのに、1.0では 0.001444。60倍。限界まで伸ばした姿勢がいちばん解きにくい。

・solver は4つの値のうち、実質2種類だった。0（既定） / 2 / 3 は小数点以下9桁までまったく同じ結果。違うのは 1 だけ。

・どちらが正確かは、条件でひっくり返る。届く距離の8割までは solver=1 が 5～100倍正確。9割を越えると逆転し、今度は既定のほうが 5～58倍正確になる。境目は 0.8 と 0.9 の間。

・だから「どちらが良い」とは言えない。腕を伸ばしきる動きが多いなら既定、曲げた姿勢で細かく合わせたいなら 1。使い方で選ぶ。

・測り方について。実験055と同じく、ジオメトリを作らずに APEX の部品だけで測っている。行列の配列を入れて、行列の配列を受け取る。シーンを組まないで、33関節でも一瞬で終わる。

次にやること

- ・関節の回る範囲を決める（configurejointlimits）
- ・APEX のグラフをジオメトリに埋め込んで持ち歩く
- ・実験060で 031～060 の振り返り点検

APEX のグラフを持ち歩く — ネットワークが、そのままジオメトリになる

2026-09-24

何を作ったか

APEX のいちばんの特徴は「ノードのネットワークそのものをデータとして持ち歩けること」だと実験050で書いた。今回それを確かめる。

グラフを `saveToGeometry` でジオメトリに落とすと、ノード1つが点1つ、配線1本がプリミティブ1つになる。6通りの大きさ（ノード3〜65個）で、点の数はノードの数と完全に一致し、プリミティブの数は配線の数と完全に一致した。

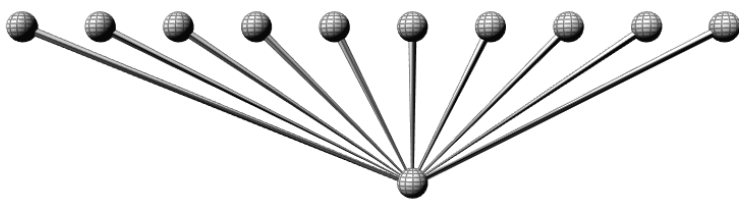
点には `name`（ノードの名前）と `callback`（どの部品か）が入っている。`Add` や `Value` という文字列が、そのまま点のアトリビュートとして並ぶ。

そして本題。1 + 2 + ... + 10 を足すグラフをファイル（.bgeo.sc、2,617バイト）に保存し、別のプロセスで読み直して実行したところ、55.0 が返った。シーンもノードも持ち込んでいない。ジオメトリのファイル1つだけである。

ファイルの重さも測った。ノード3個で 2,273バイト、65個で 3,888バイト。1ノードあたりに直すと 757.7バイトから 59.8バイトへ下がる。差から逆算すると、ノード1つの実費は約26バイトで、残りの約2,195バイトは大きさによらない土台だった。

グラフをジオメトリとして描く

1 + 2 + ... + 10 を足すグラフを、ファイルから読んでそのまま描いたもの。球が点 = ノード、線がプリミティブ = 配線。ノードの置き場所がそのまま点の位置になる。



Houdini

11点 / 10プリミティブ。上の10個が `Value`、下の1個が `Add`。これが「ネットワークがジオメトリになる」ということ。

足す数	ノード	配線	点	プリミティブ	答え	合っているか
2	3	2	3	2	3.0	はい
4	5	4	5	4	10.0	はい
8	9	8	9	8	36.0	はい
16	17	16	17	16	136.0	はい

足す数	ノード	配線	点	プリミティブ	答え	合っているか
32	33	32	33	32	528.0	はい
64	65	64	65	64	2080.0	はい

点には何が入っているか

ノード3個ぶんのグラフを書き出して、点を1つずつ読んだもの。

点	name	callback
0	add	Add
1	v0	Value
2	v1	Value
3	v2	Value

別のプロセスで読んで実行する

保存したファイルを、まったく別の hython プロセスで読み込んだ結果。

調べたこと	結果
読んだジオメトリ	11点 / 10プリミティブ
グラフとして読めたか	はい
ノード / ポート	11個 / 32個
実行した答え	55.0
期待	55 (1 + 2 + ... + 10)
一致するか	はい

グラフの大きさとファイルの重さ

同じ形のグラフで、ノードの数だけを変えて保存した。

ノード	点	バイト	1ノードあたり
3	3	2,273	757.7 B
5	5	2,425	485.0 B
9	9	2,587	287.4 B
17	17	2,756	162.1 B
33	33	3,137	95.1 B
65	65	3,888	59.8 B

わかったこと

- ・ノード1つが点1つ、配線1本がプリミティブ1つだった。6通りの大きさすべてで、点の数 = ノードの数、プリミティブの数 = 配線の数。例外なし。
- ・点には name と callback が入っている。Add<Float> や Value<Float> という部品の名前が、そのまま文字列として点に乗る。ほかに parms (パラメータ) ・ tags ・ properties ・ comment ・ Cd もある。

- ・ノードの置き場所が、そのまま点の位置になる。だからグラフを描くのに専用の道具は要らない。点に球を複製して、線を筒にすれば、それがネットワーク図になる。

- ・別のプロセスで読んでも、同じ答えが出た。ファイル1つ (2,617バイト) を渡しただけで、シーンもノードも持ち込んでいない。55.0。これが「持ち歩ける」ということ。

- ・ファイルは思ったより軽い。ノード65個で 3,888バイト。1ノードあたりは 757.7B → 59.8B と下がり続ける。

- ・その下がり方から、内訳が読める。ノード3個 2,273B、65個 3,888B。差は 1,615B で、その間にノードが62個増えている。ノード1つの実費は約26バイトで、残りの約2,195バイトは大きさによらない土台。小さいグラフほど「土台」の割合が大きい。

- ・答えも全部合っていた。3.0 / 10.0 / 36.0 / 136.0 / 528.0 / 2080.0。 $1 + 2 + \dots + n$ の式 $(n(n+1)/2)$ とすべて一致。

- ・SOP のネットワークとの違いが、ここではっきりする。SOP はシーンの中に置かれるもので、ジオメトリはその中を流れる。APEX は逆で、ネットワークのほうがジオメトリの中を流れる。だからキャラクターのリグを、モデルと一緒に1つのファイルで渡せる。

次にやること

- ・SOP から apex::invokegraph でグラフを呼ぶ
- ・骨のジオメトリとグラフを、1つのファイルにまとめる
- ・実験060で 031~060 の振り返り点検

hairgen の毛の本数は Density × 面積 − 1本は Segments + 1 点、長さは Length ちょうど

2026-09-24

何を作ったか

grid（面積 1・4）と球（半径 1）を皮にして、hairgen（ガイドなし）を Density 100・1000・5000 で生やし、毛（線）の本数・点の数・長さを数えた。

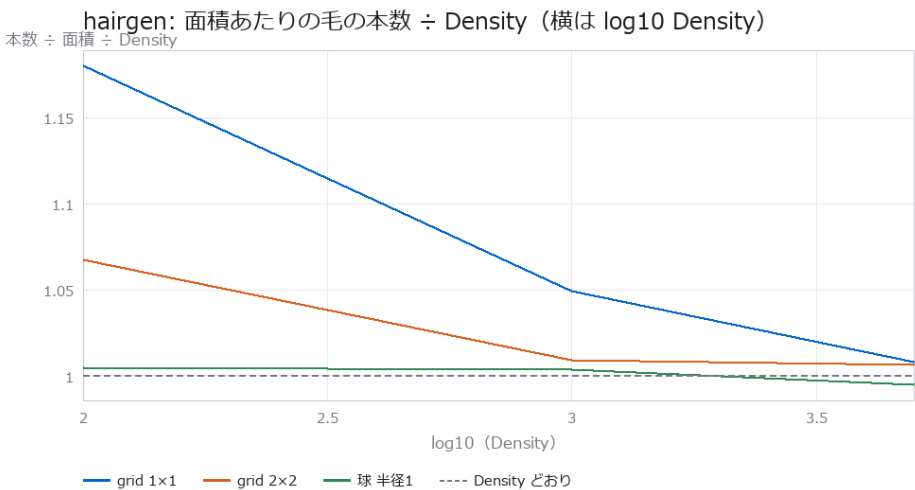
****本数 ÷ 面積 は Density に合う。***Density 5000 では grid 1×1 で 5041・grid 2×2 で 5031.25・球 半径1 で 4974.64。本数が少ないほどずれが大きく（Density 100・面積 1 で 118 本、+18%）、でたらめに撒いたときの揺れ（1/√本数、ここでは ±10%）くらいの大きさ。

****Force Count を入れると、面積によらず指定の本数ちょうど****（3通りとも 2,500 本）。面積あたりの密度は、面の広さで変わる。

****1本の毛は 9 点（既定の Segments 8 + 1）、長さは既定の Length 0.05 ちょうど。***grid ではどの毛も 0.05000、球では 0.04998 前後（ばらつき 0.00005 未満）。

****速い。***球に 62,456 本でも 0.038 秒。

皮・Density と毛



本数が多いほど 1 に近づく。

皮	面積	Density	Force Count	毛の本数	面積あたり	1本の点	長さの平均	長さの差	秒
grid 1×1	1	100	切	118	118	9	0.05000	0.00000	0.100
grid 1×1	1	1000	切	1,049	1049	9	0.05000	0.00000	0.015
grid 1×1	1	5000	切	5,041	5041	9	0.05000	0.00000	0.018
grid 1×1	1	1000	入	2,500	2500	9	0.05000	0.00000	0.016
grid 2×2	4	100	切	427	106.75	9	0.05000	0.00000	0.012
grid 2×2	4	1000	切	4,036	1009	9	0.05000	0.00000	0.017
grid 2×2	4	5000	切	20,125	5031.25	9	0.05000	0.00000	0.023
grid 2×2	4	1000	入	2,500	625	9	0.05000	0.00000	0.017

皮	面積	Density	Force Count	毛の本数	面積あたり	1本の点	長さの平均	長さの差	秒
球 半径1	12.5549	100	切	1,261	100.44	9	0.04997	0.00005	0.018
球 半径1	12.5549	1000	切	12,602	1003.75	9	0.04998	0.00003	0.023
球 半径1	12.5549	5000	切	62,456	4974.64	9	0.04998	0.00003	0.038
球 半径1	12.5549	1000	入	2,500	199.13	9	0.04998	0.00004	0.020

わかったこと

- ・毛の本数は Density × 面積で見積もれる。面積 4 の板に Density 1000 なら約 4,000 本。
- ・皮を大きくしても毛の密度を保つなら Density、本数を決めたいなら Force Count。
- ・1本の点の数は Segments + 1。点の総数は 本数 × 9 で、重さの見積もりに使える。

次にやること

- ・ガイドを使ったとき（guide から生やす）の本数
- ・Density 属性で場所ごとに変えたときの本数